Introduction To NOSQL with Cloud Database

Introduction To NOSQL with Cloud Database

DBMS, BCSE0402, Unit-5

Introduction To NOSQL with Cloud Database

B-Tech 4th Sem

All Branches

Deepa Soni
Assistant Professor
CSE-Cyber Security
Department

NIET Greater Noida
Autonomous Institute
Get Future Ready

Course Contents / Syllabus

- Introduction of NoSQL With MongoDB :
- Introduction of NoSQL Data Models,
- Overview of NoSQL Databases with their Types,
- Uses & Features of NoSQL Document Databases, CAP theorem,
- BASE Vs ACID,
- Comparison of relational databases to NoSQL stores,
- uses and deployment; - MongoDB, Cassandra, HBASE, Neo4j and Riak.
- Introduction and Features of MongoDB,
- MongoDB Operators

NIET Greater Noida
Autonomous Institute
Get Future Ready

Course Contents / Syllabus

- MongoDB Collection & Document,
- MongoDB Shell & their commands,
- CRUD operations.
- Cloud DatabaseIntroduction of Cloud Database.
- MongoDB Cloud product : Stitch, Atlas & Cloud Manager.

Branch wise Applications

- From the 1980s to the Internet era in the late 1990s, SQL databases dominated the development landscape. Large commercial applications, niche products, and custom applications of all types were based on SQL.
- But the rise of the Internet has changed application development profoundly. The amount of data, the structure of the data, the scale of applications, the way applications have developed have all changed dramatically.
- These changes have led many organizations of all sizes to adopt NoSQL database technology.
- In recent times you can easily capture and access data from various sources, like Facebook, Google, etc.
- User's personal information, geographic location data, user generated content, social graphs and machine logging data are some of the examples where data is increasing rapidly.
- To use above mentioned properties, it is necessary to process large volume of data.
- For which relational databases are not suitable. The evolution of NoSQL databases is to handle this large volume of data properly.

Prerequisite and Recap

Prerequisites:

- Linux/ Windows operating system.
- Database Management Software's such as Oracle
- Knowledge on SQL/PLSQL.
- Cloud Infrastructure.
- Handling Big Data on Unstructured Databases.
- Programming Languages (Python or Java)

Recap:

- Discussion about Cloud and Database Management System.

Topics you will learn in this unit

Introduction to Data Science:

1. Definition of NoSQL
2. History of NoSQL and Different NoSQL products
3. Exploring Mongo DB
4. Interfacing and Interacting with NoSQL
5. NoSQL Storage Architecture
6. CRUD operations with MongoDB
7. Querying, Modifying and Managing NoSQL Data stores
8. Indexing and ordering datasets (MongoDB)
9. Cloud database: - Introduction of Cloud database
10. NoSQL with Cloud Database
11. Introduction to Real time Database.

Unit Objective

The objective of the Unit is :

1.To provide an overview of an exciting growing field of NOSQL databases.

2. To inculcate the preliminary knowledge of domain of DBMS and elaborate when should NoSQL be used:

- When huge amount of data need to be stored and retrieved .
- The relationship between the data you store is not that important
- The data changing over time and is not structured.
- Support of Constraints and Joins is not required at database level
- The data is growing continuously, and you need to scale the database regular to handle the data.

Definition of NoSQL

Objective:

- **In this topic** we focus on There are several advantages of working with NoSQL databases such as MongoDB and Cassandra. The main advantages are high scalability and high availability. High scalability: NoSQL database such as MongoDB uses sharding for horizontal scaling.

Recap:

- Revision of Database Management Systems.

Definition of NoSQL

NoSQL databases (aka "not only SQL") are non-tabular databases and store data differently than relational tables. NoSQL databases come in a variety of types based on their data model. The main types are document, key-value, wide-column, and graph. They provide flexible schemas and scale easily with large amounts of data and high user loads.

When people use the term “NoSQL database,” they typically use it to refer to any non-relational database. Some say the term “NoSQL” stands for “non SQL” while others say it stands for “not only SQL.” Either way, most agree that NoSQL databases are databases that store data in a format other than relational tables.

A NoSQL (originally referring to "non-SQL" or "non-relational") database provides a mechanism for storage and retrieval of data that is modeled in means other than the tabular relations used in relational databases. Such databases have existed since the late 1960s, but the name "NoSQL" was only coined in the early 21st century, triggered by the needs of Web 2.0 companies NoSQL databases are increasingly used in big data and real-time web applications. NoSQL systems are also sometimes called Not only SQL to emphasize that they may support SQL-like query languages or sit alongside SQL databases in polyglot-persistent architectures

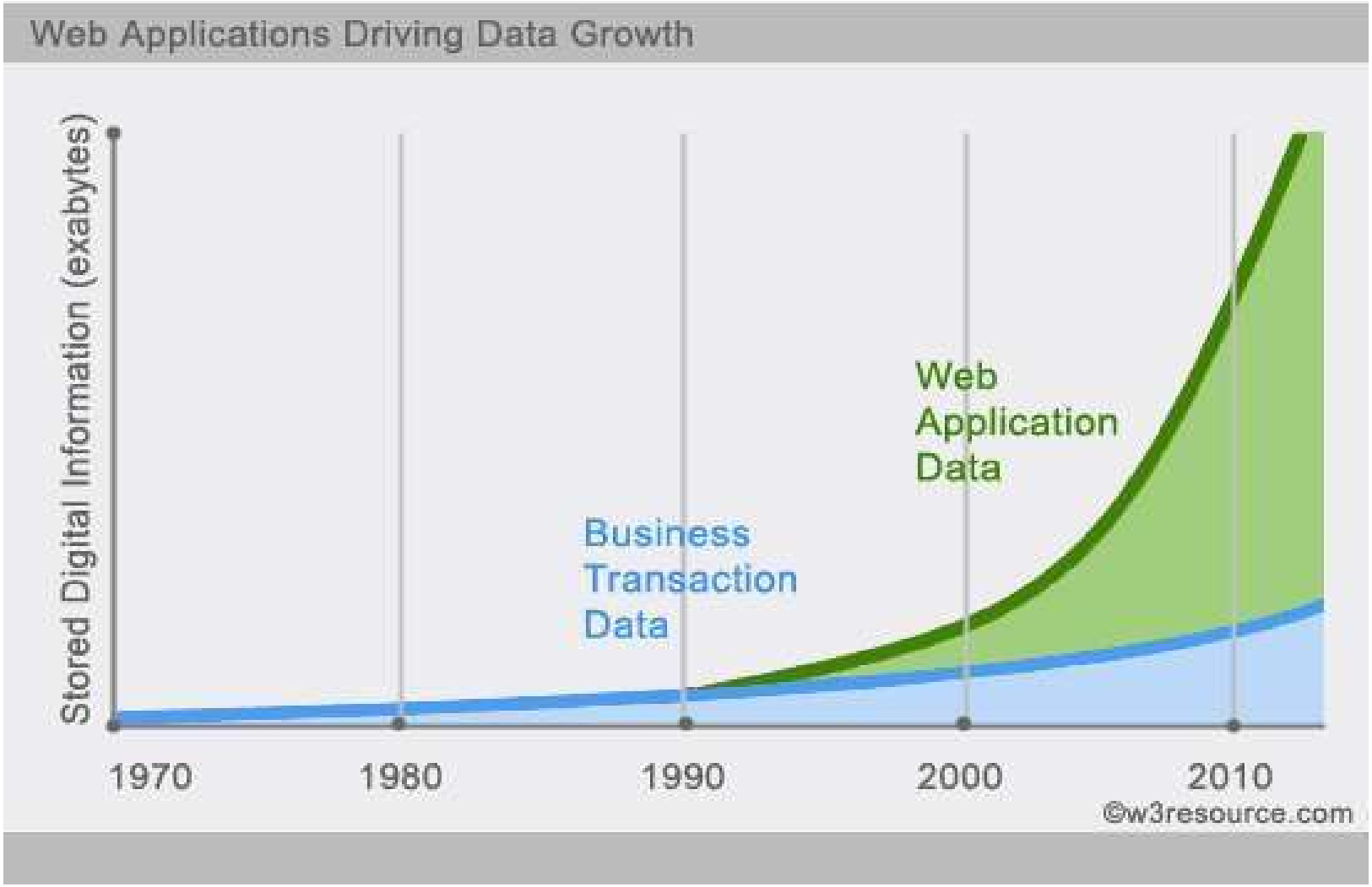
Definition of NoSQL

Introduction NoSQL

1. Stands for Not Only SQL.
2. The idea of NoSQL founded in 1998 with term lightweight Schema Less by Carlo Strozzi.
3. Open-source database.
4. NoSQL will be the future database.
5. Very compatible with distributed systems.
6. Lower cost.
7. High performance database.
8. Founded to handle huge data space.
9. Used by Facebook , Google , Wikipedia …

Definition of NoSQL

Increasing Web Application Data.



Definition of NoSQL

Characteristic of NoSQL

1. Large data volumes.
2. Scalable replication and distribution (Horizontal scaling).
3. Queries need to return answers quickly.
4. Asynchronous Inserts & Updates.
5. Schema-less.
6. Designed to support Caching without 3-rd party tools.
7. ACID transaction properties are not needed.
8. BASE here.
9. CAP Theorem.
10. No Joins statement.
11. No complicated Relationships
12. Less administration time(less cost).

Definition of NoSQL

Major NoSQL Types

- **Key-Value Store.**
  - Hashing.
  - Basic get/put/delete.
  - Crazy fast because there is key to get set of values.
- **Document Store.**
  - JSON,XML … document structured.
  - No Join.(handle it in your code).
- **Column Database.**
  - Each storage block contains data from only one column.
  - Reduce access and scanning time.
  - Still use tables without joins statements.
  - Better for data analytics.

Definition of NoSQL

Developers Viewpoint !

SQL is better ?

- Natural reaction.
- Everyone's experience.
- Fear of change.

NoSQL Will :

- Simplify your data model.
- Easy to install.
- your bugs will be fewer and easier to find.
- Lower administration / less DBAs.
- performance is going to be awesome.
- Scale will be much simpler.
- Rapid Development.
- Large binary objects.
- Graphs/relationships.

Comparison with RDBMS

Feature	NoSQL	RDBMS
Schema	Dynamic	Fixed
Scalability	Horizontal	Vertical
Transactions	BASE	ACID
Data Model	Key-Value, Document, Column, Graph	Table-based

Definition of NoSQL

Summary

NoSQL :

- Handle huge data.
- High availability with small cost.
- More data redundancy.
- High performance.
- Less administration time.
- Less standards.

Pick the right tool for your job !

SQL :

- Good to solve ACID problems.
- Expensive.
- Less data redundancy.
- Increasing availability mean increasing cost.
- More standards.
- More administration.

Uses & Features of NoSQL

NoSQL database features

Each NoSQL database has its own unique features. At a high level, many NoSQL databases have the following features:

- Flexible schemas
- Horizontal scaling
- Fast queries due to the data model
- Ease of use for developers

Uses & Features of NoSQL

1. Flexible Schemas

•**What It Means**: Unlike traditional relational databases that require predefined schemas (structured tables with fixed columns), NoSQL databases support dynamic schemas. You can store data without defining a strict structure beforehand.

•**Benefits**:

- Accommodates changes easily when the application evolves.
- Useful for applications with variable or hierarchical data (e.g., JSON, XML).
- Reduces downtime when the schema needs to be updated.

•**Use Case**: In e-commerce, product attributes (like size, color, weight) can vary significantly. NoSQL databases allow for flexible attributes for each product.

Uses & Features of NoSQL

2. Horizontal Scaling

•**What It Means**: Horizontal scaling involves adding more servers (nodes) to distribute the database workload, as opposed to vertical scaling, which upgrades a single server's hardware.

•**Benefits**:

- Scalability to handle massive amounts of data and traffic.
- Cost-efficient because adding more servers is often cheaper than upgrading a single powerful server.
- Enables high availability and fault tolerance through replication across nodes.

•**Use Case**: Social media platforms like Facebook or Instagram use horizontal scaling to handle billions of users and their interactions in real time.

Uses & Features of NoSQL

3. Fast Queries (Optimized for the Data Model)

•**What It Means**: NoSQL databases are designed to store and retrieve data in a way that minimizes query time. They use data models (key-value, document, graph, or column-family) that fit specific application needs.

•**Benefits**:

- Faster data access compared to traditional relational models, especially for non-relational data.
- Avoids the overhead of SQL joins and complex transactions.
- Enables efficient handling of specific query patterns.

•**Use Case**: In a caching system, key-value stores like Redis or Memcached provide extremely fast read and write operations.

Uses & Features of NoSQL

4. Ease of Use for Developers

•**What It Means**: NoSQL databases often provide intuitive APIs and data storage approaches that align with modern programming practices.

•**Benefits**:

- Simplifies development by using natural data formats like JSON or BSON.
- Reduces the complexity of mapping objects in code to database tables (common in relational databases).
- Streamlined integration with modern frameworks and tools.

•**Use Case**: Applications using JavaScript-based frameworks (e.g., Node.js) can easily integrate with document-based NoSQL databases like MongoDB, as they both use JSON

Uses & Features of NoSQL

Uses of NoSqL

NoSQL databases are widely used across various industries for applications requiring scalability, flexibility, and high performance. Here are the primary **uses of NoSQL databases**:

1. Big Data Applications

•**Why**: NoSQL databases are designed to handle massive volumes of unstructured or semi-structured data generated at high velocity.

•**Examples**:

- Storing logs, metrics, and event data for monitoring and analytics.
- Managing clickstream data in web and mobile applications.

2. Real-Time Applications

•**Why**: NoSQL databases provide low-latency read and write operations, essential for real-time interactions.

•**Examples**:

- Chat applications (e.g., WhatsApp or Slack).
- Online multiplayer games that require real-time updates.
- Financial systems for processing live transactions.

Uses & Features of NoSQL

3. Content Management Systems (CMS)

•**Why**: NoSQL databases offer flexibility to store diverse types of content such as text, images, videos, and metadata.

•**Examples**:

- Storing and managing user-generated content on platforms like YouTube or Instagram.
- Building scalable CMS for blogs, news, or e-commerce websites.

4. Internet of Things (IoT)

•**Why**: IoT devices generate continuous streams of sensor data, which need to be processed, stored, and queried efficiently.

•**Examples**:

- Storing telemetry data from smart devices (e.g., smart meters, fitness trackers).
- Managing real-time data for industrial IoT applications.

Uses & Features of NoSQL

5. E-commerce

•**Why**: E-commerce platforms require flexible schemas to handle varying product attributes, scalability to support high traffic, and fast queries for user searches.

•**Examples**:

- Product catalog management (e.g., storing product details in MongoDB or DynamoDB).
- User session storage for a seamless shopping experience.

6. Social Media and Networking

•**Why**: Social media platforms generate vast amounts of interconnected, semi-structured data that require horizontal scaling and graph-based queries.

•**Examples**:

- Storing user profiles and posts.
- Tracking and analyzing user relationships and activities using graph databases (e.g., Neo4j).

Uses & Features of NoSQL

7. Recommendation Systems

•**Why**: NoSQL databases can store and retrieve user preferences, browsing histories, and interaction patterns efficiently.

•**Examples**:

- Powering recommendation engines for e-commerce (e.g., Amazon, Flipkart).
- Streaming services like Netflix and Spotify for personalized suggestions.

8. Mobile and Web Applications

•**Why**: Modern mobile and web applications require databases that are scalable, fast, and easy to integrate with JSON-based APIs.

•**Examples**:

- Storing user profiles and app data in document stores like MongoDB.
- Session management and real-time notifications.

9. Gaming

•**Why**: Gaming platforms need real-time data access for player stats, leaderboards, and multiplayer synchronization.

•**Examples**:

- Managing game state and progress in online multiplayer games.
- Storing in-game purchases and virtual assets.

Uses & Features of NoSQL

10. Search Engines

•**Why**: NoSQL databases support fast, full-text search capabilities and structured indexing.

•**Examples**:

- Elasticsearch for building search functionality in websites and applications.
- Managing user queries and logs for search optimization.

11. Healthcare and Biomedicine

•**Why**: NoSQL databases can handle large-scale, complex, and diverse medical datasets, including patient records, diagnostic images, and sensor data.

•**Examples**:

- Storing Electronic Health Records (EHRs).
- Real-time analysis of medical data for decision support systems.

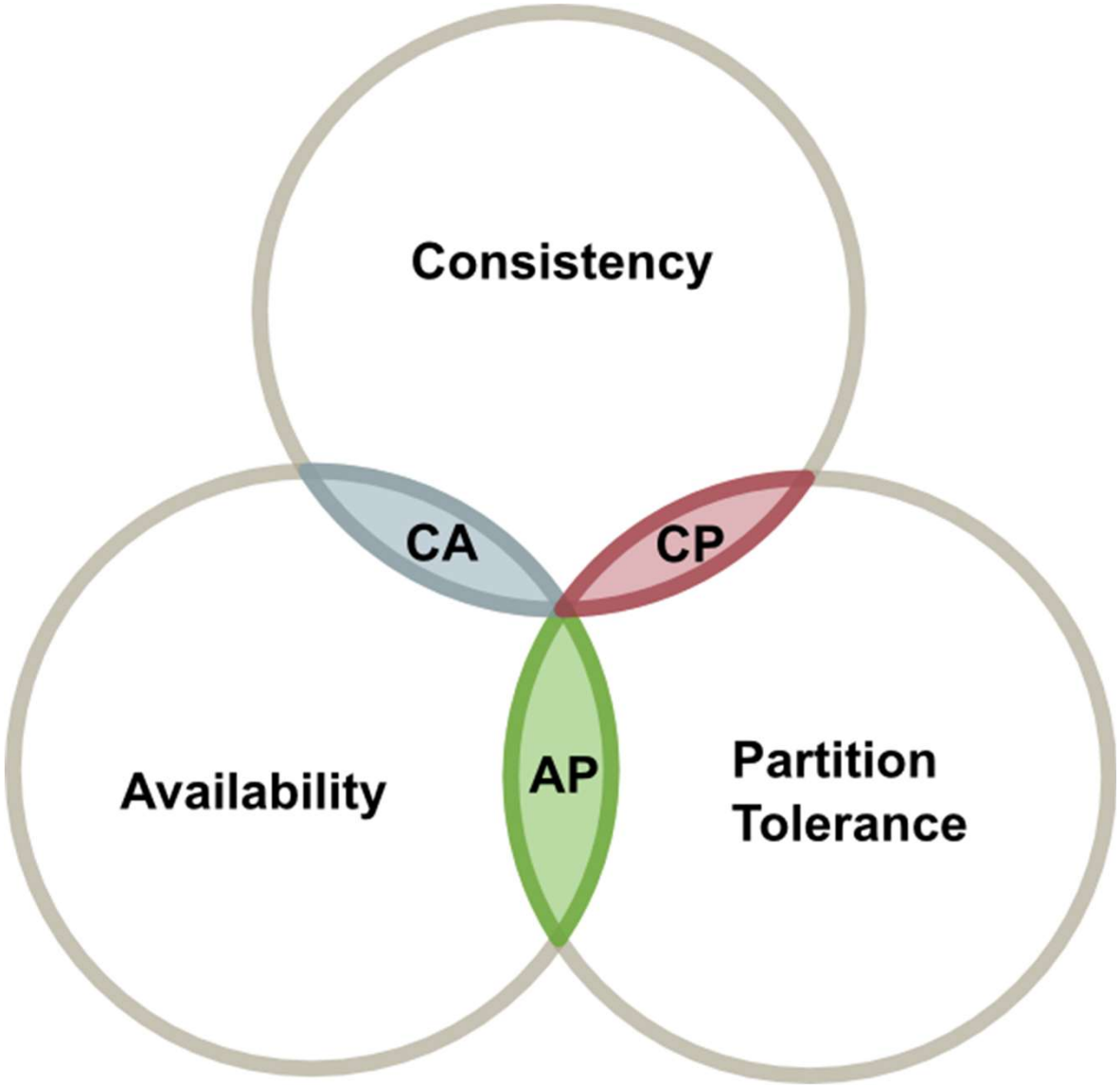
12. Distributed Systems and Microservices

•**Why**: NoSQL databases are ideal for microservices architectures due to their scalability and distributed nature.

•**Examples**:

- Storing data specific to individual services (e.g., user authentication, payment systems).
- Maintaining event logs and distributed caches.

CAP Theorem



CAP Theorem

- **Consistency:** Ensures all nodes have the latest data.
- **Availability:** Guarantees that every request receives a response.
- **Partition Tolerance:** Functions despite network failures.

- **Trade-offs are necessary**: A distributed system can have at most two out of the three properties.
- **CP (Consistency + Partition Tolerance)**: Prioritizes data accuracy but may sacrifice availability during network failures (e.g., **HBase, MongoDB in strict mode**).
- **AP (Availability + Partition Tolerance)**: Ensures responses but may return stale or inconsistent data (e.g., **Cassandra, DynamoDB**).
- **CA (Consistency + Availability)**: Works only in ideal conditions without network partitions (practically impossible in a distributed system).

BASE Vs ACID

ACID

1. Atomic – All transaction completes (commit) or none of it completes.
2. Consistent – Consistency is defined in terms of constraints.
3. Isolated – The transaction will behave as if it is the only operation being performed upon the database
4. Durable – Upon completion of the transaction, the operation will not be reversed.

BASE Vs ACID

BASE

1. Basically Available.
2. Soft state(expiration of information).
3. Eventually Consistent.
4. Weak consistency.
5. Availability first.
6. Simpler and faster.

BASE Vs ACID

- **ACID (Atomicity, Consistency, Isolation, Durability)**: Ensures transaction reliability (used in RDBMS).
- **BASE (Basically Available, Soft state, Eventually consistent)**: Prioritizes availability and performance (used in NoSQL).
- BASE is suitable for real-time applications with distributed data needs.

Comparison of Relational Databases (RDBMS) vs. NoSQL Stores

Relational Database	NoSQL
It is used to handle data coming in low velocity.	It is used to handle data coming in high velocity.
It gives only read scalability.	It gives both read and write scalability.
It manages structured data.	It manages all type of data.
Data arrives from one or few locations.	Data arrives from many locations.
It supports complex transactions.	It supports simple transactions.
It has single point of failure.	No single point of failure.
It handles data in less volume.	It handles data in high volume.
Transactions written in one location.	Transactions written in many locations.
support ACID properties compliance	doesn't support ACID properties
Its difficult to make changes in database once it is defined	Enables easy and frequent changes to database
schema is mandatory to store the data	schema design is not required
Deployed in vertical fashion.	Deployed in Horizontal fashion.

Comparison of Relational Databases (RDBMS) vs. NoSQL Stores

When to Use RDBMS vs. NoSQL?

RDBMS when:

- Data structure is well-defined and relational.
- ACID transactions are critical (e.g., banking, inventory management).
- Complex queries and reporting are needed.

NoSQL when:

- Scalability and performance are top priorities (e.g., social media, real-time analytics).
- Data schema is evolving or flexible.
- Handling large volumes of unstructured or semi-structured data.

MongoDB: Uses and Deployment

Uses of MongoDB

MongoDB is a **document-oriented NoSQL database**, designed for flexibility, scalability, and high performance. It is widely used in applications requiring **rapid development and handling of large-scale unstructured or semi-structured data**.

Common Use Cases:

- **Content Management Systems (CMS)** – Websites, blogs, and e-commerce platforms (e.g., Shopify, Magento).
- **Real-time Analytics** – IoT, stock market tracking, and user behavior analysis.
- **Big Data Applications** – Processing large volumes of log data, social media analytics.
- **Internet of Things (IoT)** – Storing sensor data and event-driven architectures.
- **Mobile & Web Applications** – User profiles, session storage, chat applications.
- **Gaming & Leaderboards** – Storing game states, player profiles, and leaderboards.

MongoDB: Uses and Deployment

Deployment Options for MongoDB

MongoDB supports multiple deployment strategies, including **on-premises, cloud, and hybrid** solutions.

A. Self-Managed Deployment (On-Premises or Cloud VM)

Install MongoDB on **physical servers, VMs, or Docker containers**.
Requires manual **sharding, replication, and scaling** for large-scale applications.
Used when **data security, compliance, or cost control** is a priority.
Tools: **MongoDB Community Edition, MongoDB Enterprise Advanced**.

B. Managed Cloud Deployment (MongoDB Atlas)

Fully managed, cloud-based MongoDB service (AWS, GCP, Azure).
Automatic **scaling, backups, monitoring, and security**.
Best for **startups and enterprises needing hassle-free scalability**.
Supports **global clusters and multi-region replication**.

MongoDB: Uses and Deployment

C. Kubernetes & Containerized Deployment
Deploy MongoDB in **Kubernetes clusters** for **microservices** architectures.
Uses **MongoDB Kubernetes Operator** for auto-scaling and fault tolerance.
Suitable for **DevOps-driven cloud-native applications**.

D. Hybrid & Multi-Cloud Deployment
Deploy MongoDB across multiple cloud providers or hybrid environments.
Ensures **data redundancy, failover protection, and regulatory compliance**.
Useful for **enterprises with strict data sovereignty laws**.

MongoDB: Uses and Deployment

Key Features for Deployment

- **Replication**: Ensures high availability with **Replica Sets**.
- **Sharding**: Distributes data across multiple nodes for scalability.
- **Load Balancing**: Handles large read/write workloads efficiently.
- **Security**: Supports **TLS/SSL encryption, authentication, and role-based access control (RBAC)**.

Casandra: Uses and Deployment

Uses of Cassandra

Apache Cassandra is a **distributed NoSQL database** designed for **high availability, fault tolerance, and scalability**. It follows a **peer-to-peer architecture** and is optimized for handling **large-scale, high-velocity data**.

Common Use Cases:

- **Real-Time Big Data Applications** – Logs, metrics, sensor data, IoT devices.
- **High-Throughput Transactional Applications** – Banking, fraud detection, stock trading.
- **Social Media and Messaging Apps** – Facebook, Twitter, Discord, WhatsApp.

Casandra: Uses and Deployment

- **E-commerce & Retail** – Product catalogs, recommendations, and inventory management.
- **Streaming & Time-Series Data** – IoT event tracking, telemetry, clickstream analysis.
- **Personalization & Recommendation Engines** – AI-driven user suggestions.
- **Decentralized Applications (DApps)** – Blockchain and Web3 infrastructure.

Why Use Cassandra?

Masterless Architecture → No single point of failure.
Horizontal Scalability → Easily scales by adding more nodes.
High Availability → Designed for **multi-region replication**.
Optimized for Writes → Handles high write loads efficiently.
Tunable Consistency → Adjust **eventual or strong consistency** per use case.

Casandra: Uses and Deployment

Deployment Options for Cassandra

A. Self-Managed Deployment (On-Premise or Cloud VM)
Install on **bare metal, VMs, or Docker containers**.
Requires **manual setup of replication, sharding, and monitoring**.
Suitable for enterprises needing **complete control** over infrastructure.
Tools: **Apache Cassandra, ScyllaDB (low-latency alternative), Instaclustr for managed setup**.

B. Managed Cloud Services (Database-as-a-Service)

Datastax Astra → Fully managed **Cassandra-as-a-Service** on AWS, GCP, Azure.
Amazon Keyspaces (for Cassandra) → Cassandra-compatible service on AWS.
Azure Managed Instance for Cassandra → Cassandra on Microsoft Azure.
Best for **fast deployment, automated scaling, and maintenance-free management**.

Casandra: Uses and Deployment

C. Kubernetes & Containerized Deployment
Cassandra Kubernetes Operator enables deployment on **K8s clusters**.
Ideal for **cloud-native microservices** architectures.
Works well with **Docker and Helm Charts** for DevOps-driven environments.

D. Multi-Cloud & Hybrid Deployment
Cassandra's **multi-data center replication** ensures **geo-distributed storage**.
Used in industries requiring **data sovereignty and disaster recovery**.
Can replicate across **AWS, GCP, Azure, or on-prem data centers**.

Casandra: Uses and Deployment

Key Features for Deployment

- **Peer-to-Peer Architecture**: Every node is equal → no master-slave bottlenecks.
- **Partitioning & Sharding**: Uses **consistent hashing** for automatic distribution.
- **Multi-Region & Multi-Data Center Replication**: Ensures **zero downtime**.
- **Tunable Consistency Levels**: Choose between **strong, quorum, or eventual consistency**.
- **Fault Tolerance**: Survives node failures without service disruption.
- **High-Performance Writes**: Ideal for **time-series, logs, and analytics workloads**.

HBase: Uses and Deployment

Uses of Hbase

HBase is a **distributed, column-family NoSQL database** built on top of **Hadoop and HDFS**. It is designed for **real-time read/write access to large datasets** and supports **horizontal scaling**. Unlike traditional relational databases, HBase is optimized for **sparse, unstructured, and semi-structured data**.

Common Use Cases:

- **Big Data Analytics** – Large-scale data processing using Hadoop.
- **Time-Series Data Storage** – Sensor data, event logs, IoT data streams.
- **Real-Time Data Processing** – Clickstream analysis, fraud detection.
- **Search Engines & Indexing** – Scalable indexing for text search engines.
- **Data Warehousing** – High-volume, structured storage with quick retrieval
- **Recommendation Systems** – AI-driven personalization for e-commerce, streaming services.
- **Financial Services** – Stock market analytics, risk analysis.
  Government & Research – Genomic data processing, satellite image storage.

HBase: Uses and Deployment

Why Use HBase?

Scalability → Handles **petabytes of data** with ease.
High Throughput → Optimized for **fast reads and writes**.
Hadoop Integration → Works natively with **HDFS, Spark, Hive, and MapReduce**.
Column-Oriented Storage → Efficient storage for wide datasets.
Automatic Sharding → Data is automatically partitioned across nodes.

HBase: Uses and Deployment

Deployment Options for Cassandra

A.Self-Managed Deployment (On-Premise or Cloud VM)

Install on **Hadoop clusters** using **Apache HBase**.
Requires manual configuration of **HDFS, Zookeeper, and Region Servers**.
Used in **data centers or private cloud environments** for full control.
Monitoring tools: Grafana, Prometheus, Apache Ambari.

B. Managed Cloud Services (HBase-as-a-Service)

Amazon DynamoDB (HBase-compatible alternative) → Fully managed, NoSQL with auto-scaling.
Google Bigtable → Managed HBase-compatible cloud database.
Azure Cosmos DB (Table API) → Cloud-native HBase-like service.
Best for **reducing operational complexity** and **leveraging cloud scalability**.

HBase: Uses and Deployment

C. Kubernetes & Containerized Deployment
Deploy using **HBase Kubernetes Operator** for cloud-native applications.
Works with **Docker and Helm Charts** to simplify scaling.
Best for **DevOps-driven environments and hybrid cloud** setups.

D. Hybrid & Multi-Cloud Deployment
HBase supports **multi-cluster replication** for **cross-region** data availability.
Useful for **disaster recovery, data sovereignty, and global applications**.
Can be deployed across **AWS, GCP, Azure, or hybrid environments**

HBase: Uses and Deployment

Key Features for Deployment

- **Column-Family Storage**: Efficiently stores **wide and sparse datasets**.
- **Strong Consistency**: Ensures **ACID properties** at the row level.
- **Automatic Partitioning**: Uses **Region Servers** to distribute data across nodes.
- **HDFS Integration**: Stores massive amounts of data efficiently.
- **Fault Tolerance**: Relies on **Zookeeper** for cluster coordination.
- **Batch & Stream Processing**: Works with **Apache Spark, Flink, and Storm**

Neo4j: Uses and Deployment

Uses of Neo4j

Neo4j is a **graph database** designed for **highly connected data** and complex relationships. Unlike relational and NoSQL databases, it **stores and queries data as nodes, edges, and properties**, making it ideal for **graph traversal** and **relationship-heavy applications**.

Common Use Cases:

- **Social Networks** – Analyzing user connections (e.g., LinkedIn, Facebook).
- **Fraud Detection** – Identifying suspicious transactions using pattern recognition.
- **Recommendation Engines** – AI-driven suggestions for e-commerce, streaming, and news.
- **Knowledge Graphs** – Building **semantic web applications** (e.g., Google's Knowledge Graph).
- **Network & IT Infrastructure Management** – Analyzing dependencies in **telecom and cloud** networks.
- **Supply Chain Optimization** – Managing logistics and real-time inventory tracking.
- **Cybersecurity** – Threat analysis, attack path visualization.
- **Healthcare & Genomics** – Mapping disease relationships and DNA sequencing.

Neo4j: Uses and Deployment

Why Use Neo4j?

Graph-Based Storage → Efficient for **highly connected data**.

Index-Free Adjacency → Fast relationship traversal compared to SQL JOINs.

Cypher Query Language (CQL) → Intuitive and optimized for graph queries.

ACID-Compliant Transactions → Ensures data consistency.

Scalability → Supports horizontal scaling with **Neo4j Fabric**.

Neo4j: Uses and Deployment

Deployment Options for Neo4j

A. Self-Managed Deployment (On-Premise or Cloud VM)
Install **Neo4j Community Edition** (free) or **Neo4j Enterprise Edition** (paid).
Deploy on **bare metal, VMs, or Docker containers**.
Requires manual **clustering, replication, and failover management**.
Suitable for enterprises needing **full control over data security and infrastructure**.

B. Managed Cloud Deployment (Neo4j AuraDB)
Neo4j AuraDB – Fully managed **graph database-as-a-service** (GCP, AWS, Azure).
Automatic **backup, scaling, and monitoring**.
Ideal for **startups and enterprises needing hassle-free graph database management**.

Neo4j: Uses and Deployment

C. Kubernetes & Containerized Deployment
Deploy **Neo4j using Kubernetes Operator** for **microservices architectures**.
Works well with **Docker and Helm Charts** for automated scaling.
Best for **cloud-native applications requiring flexible scaling**.

D. Multi-Cloud & Hybrid Deployment
Neo4j Fabric enables **distributed graph processing across multiple clusters**.
Supports hybrid deployments across **on-premise and cloud environments**.
Used in industries requiring **geo-distributed graph processing** (e.g., banking, telecom).

Neo4j: Uses and Deployment

Key Features for Deployment

- **Graph Model Storage**: Stores data as **nodes and relationships** instead of tables.
- **Cypher Query Language (CQL)**: Simple yet powerful for querying complex relationships.
- **ACID Compliance**: Ensures **data integrity and consistency**.
- **Horizontal Scaling with Sharding**: Neo4j **Fabric** enables large-scale distributed graphs.
- **Multi-Master Clustering**: Supports **high availability** and **fault tolerance**.
- **Graph Data Science (GDS)**: Built-in **graph algorithms for machine learning**.

Riak: Uses and Deployment

Uses of Riak

Riak is a **distributed NoSQL key-value store** designed for **high availability, fault tolerance, and scalability**. It is built on **eventual consistency** and follows an **AP (Availability & Partition Tolerance) model** of the CAP theorem. Riak is optimized for **large-scale, decentralized applications** and workloads requiring **low-latency, high-throughput storage**.

Common Use Cases:

Distributed Storage – Large-scale, highly available storage solutions.
Internet of Things (IoT) – Handling massive amounts of sensor data.
Session Management – Storing user sessions for high-traffic applications.
E-commerce & Retail – Product catalogs, recommendation engines, and transaction logs.
Messaging & Chat Applications – Fast, scalable message storage.
Financial Services & Banking – Fraud detection, real-time analytics.
Log Management & Analytics – Storing large-scale system logs.
Content Delivery Networks (CDN) – Caching and distributing content globally.

Riak: Uses and Deployment

Why Use Riak?

High Availability → Ensures **data access even during node failures**.

Horizontal Scalability → Expands seamlessly by adding new nodes.

Tunable Consistency → Supports both **eventual and strong consistency** per use case.

Multi-Model Support → Key-value, time-series, and object storage (Riak KV, Riak TS, Riak S2).

Multi-Datacenter Replication → Supports **geo-distributed deployments**.

.

Riak: Uses and Deployment

Deployment Options for Riak

A. Self-Managed Deployment (On-Premise or Cloud VM)
Install **Riak KV (Key-Value Store)** or **Riak S2 (Object Storage)** on **bare metal or VMs**.

Requires manual **clustering, replication, and tuning**.
Used by enterprises requiring **full control over data sovereignty and security**.
Monitoring tools: **Riak Control (Web UI), Prometheus, Grafana**.

B. Managed Cloud Deployment

Riak CS (Cloud Storage) – Amazon S3-compatible object storage solution.
AWS, GCP, Azure – Deploy Riak on cloud VMs for distributed workloads.
Suitable for companies needing **scalable, high-availability key-value storage**.

Riak: Uses and Deployment

C. Kubernetes & Containerized Deployment

Riak Kubernetes Operator enables deployment on **K8s clusters**.

Works well with **Docker and Helm Charts** for automated deployment and scaling.

Ideal for **cloud-native microservices** architectures.

D. Multi-Cloud & Hybrid Deployment

Supports **multi-datacenter replication** for **disaster recovery** and **high availability**.

Can be deployed across **on-premise and cloud environments** for redundancy.

Best for global-scale applications requiring **geo-distributed data consistency**.

Riak: Uses and Deployment

Key Features for Deployment

Dynamo-Inspired Architecture: Uses **consistent hashing** for automatic data distribution.
Eventual Consistency with Tunable Consistency: Configurable per application needs.
Multi-Datacenter Replication: Supports **global-scale** deployments.
Fault Tolerance: Self-healing nodes ensure **zero downtime**.
Riak S2 (Object Storage): Alternative to **Amazon S3** for private cloud storage

Exploring Mongo DB

Objective:

- **In this topic** we focus on MongoDB which is a source-available cross-platform document-oriented database program. Classified as a NoSQL database program, MongoDB uses JSON-like documents with optional schemas. MongoDB is developed by MongoDB Inc. and licensed under the Server Side Public License.
- **Recap:**
- Revision of Database Management Systems.

NoSQL Databases: Introduction to NoSQL & MongoDB

About MongoDB

- MongoDB is an open-source document database and leading NoSQL database. MongoDB is written in C++.
- This study will give you great understanding on MongoDB concepts needed to create and deploy a highly scalable and performance-oriented database.

NoSQL Databases: Introduction to NoSQL & MongoDB

MongoDB

- MongoDB is a cross-platform, document oriented database that provides, high performance, high availability, and easy scalability. MongoDB works on concept of collection and document.
- **Database**
- Database is a physical container for collections. Each database gets its own set of files on the file system. A single MongoDB server typically has multiple databases.
- **Collection**
- Collection is a group of MongoDB documents. It is the equivalent of an RDBMS table. A collection exists within a single database. Collections do not enforce a schema. Documents within a collection can have different fields. Typically, all documents in a collection are of similar or related purpose.
- **Document**
- A document is a set of key-value pairs. Documents have dynamic schema. Dynamic schema means that documents in the same collection do not need to have the same set of fields or structure, and common fields in a collection's documents may hold different types of data.
- The following table shows the relationship of RDBMS terminology with MongoDB.

NoSQL Databases: Introduction to NoSQL & MongoDB

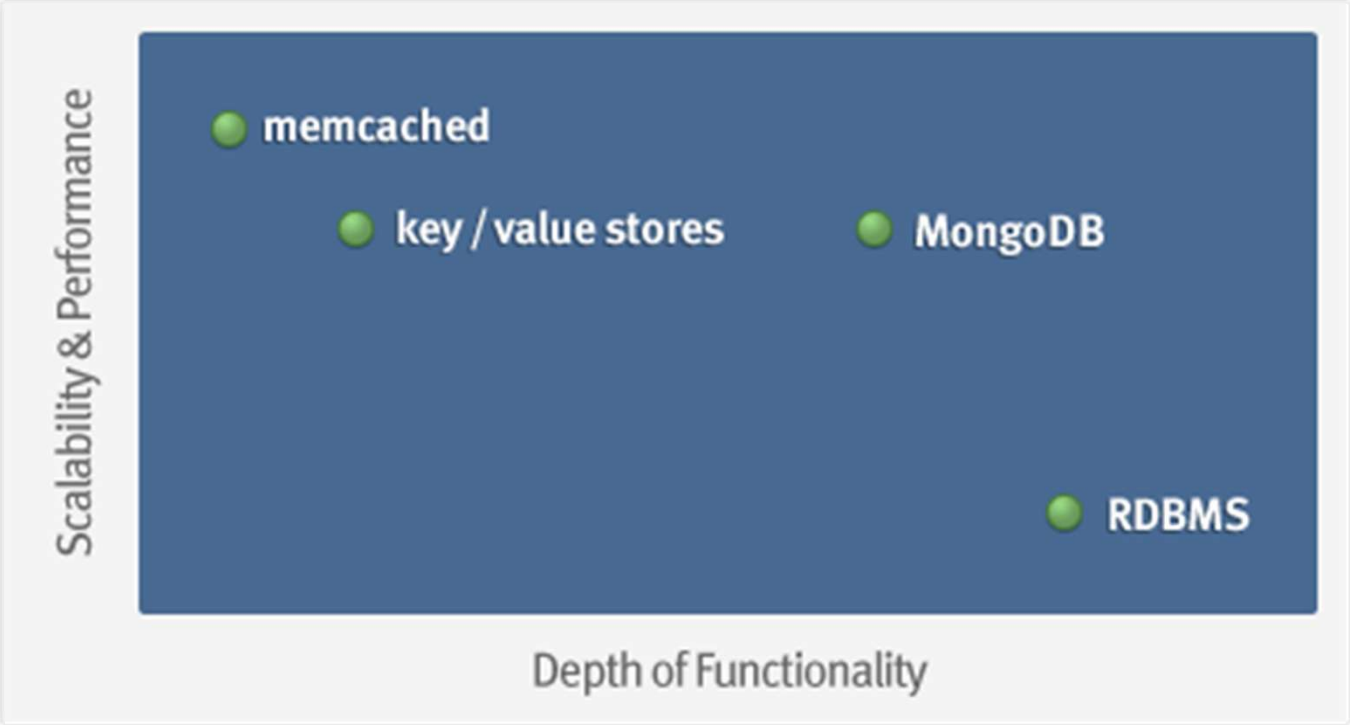
RDBMS	MongoDB
Database	Database
Table	Collection
Tuple/Row	Document
column	Field
Table Join	Embedded Documents
Primary Key	Primary Key (Default key _id provided by mongodb itself)
Database Server and Client	
Mysqld/Oracle	mongod
mysql/sqlplus	mongo

Introduction MongoDB

- Name comes from “Hu**mongo**us” & huge data
- Written in C++, developed in 2009
- Creator: 10gen, former doublick

MongoDB: Goal

- Goal: bridge the gap between key-value stores (which are fast and scalable) and relational databases (which have rich functionality).



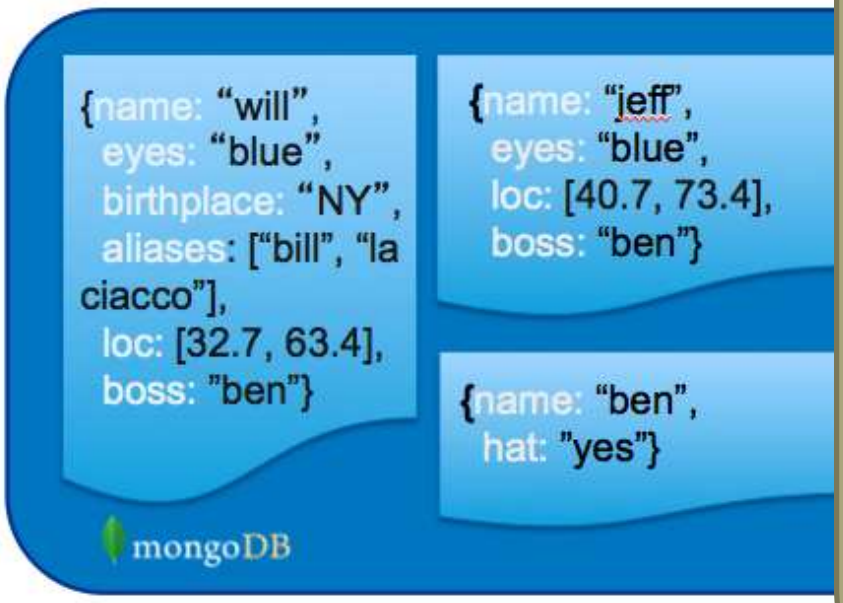
Introduction MongoDB

What is MongoDB?

- **Defination:** MongoDB is an **open source**, **document-oriented** database designed with both scalability and developer agility in mind.
- Instead of storing your data in tables and rows as you would with a relational database, in MongoDB you store **JSON-like documents** with **dynamic schemas (schema-free, schemaless)**.

What is MongoDB? (Cont'd)

- **Document-Oriented DB**
  - Unit object is a document instead of a row (tuple) in relational DBs

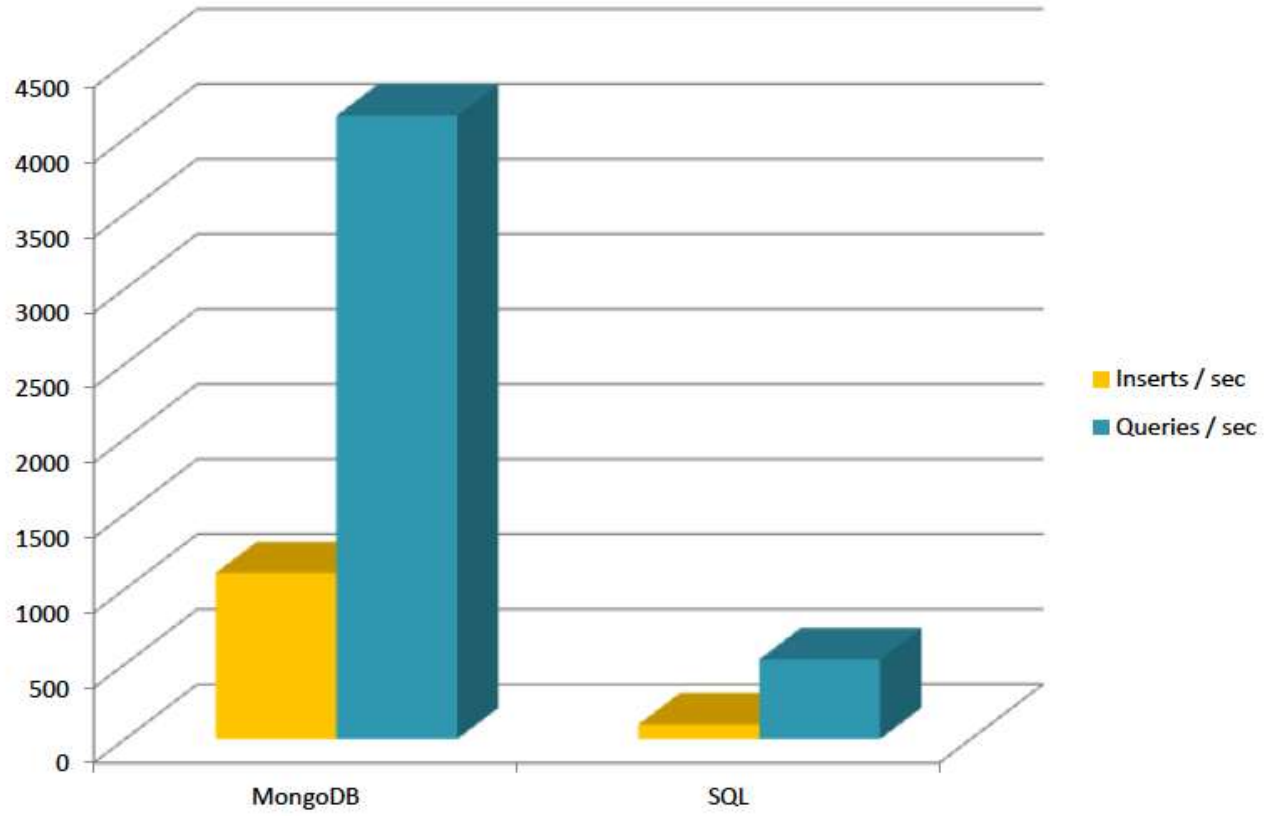



```
> db.user.findOne({age:39})
{
        "_id" : ObjectId("5114e0bd42…"),
        "first" : "John",
        "last" : "Doe",
        "age" : 39,
        "interests" : [
                "Reading",
                "Mountain Biking ]
        "favorites": {
                "color": "Blue",
                "sport": "Soccer"}
}
```

Introduction MongoDB

Is It Fast?

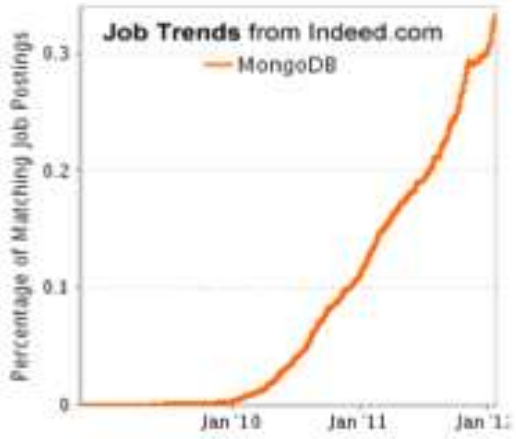
- For semi-structured & complex relationships: Yes



Introduction MongoDB

It is Growing Fast

#2 ON INDEED'S FASTEST GROWING JOBS



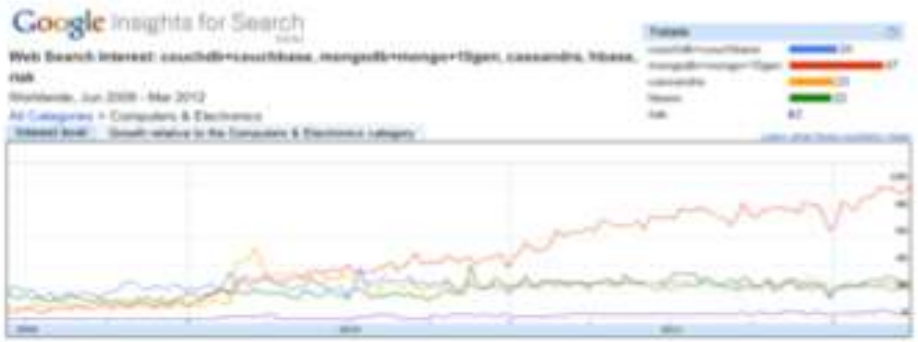
Top Job Trends

1. HTML5
2. **MongoDB**
3. iOS
4. Android
5. Mobile app
6. Puppet
7. Hadoop
8. jQuery
9. PaaS
10. Social Media

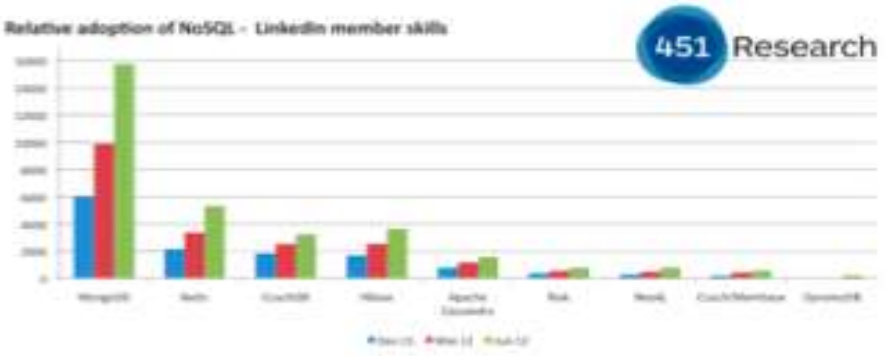
JASPERSOFT BIGDATA INDEX

Demand for MongoDB, the document-oriented NoSQL database, saw the biggest spike with over 200% growth in 2011.

GOOGLE SEARCHES



451 GROUP "MONGODB INCREASING ITS DOMINANCE"



Introduction MongoDB

Integration with Others

- C
- C++
- Erlang
- Haskell
- Java
- Javascript
- .NET (C# F#, PowerShe
- Node.js
- Perl
- PHP
- Python
- Ruby
- Scala

NoSQL: Categories

- Key-value

- Graph database

- Document-oriented

- Column family

Interfacing and Interacting with NoSQL

Objective:

- **In this topic** we focus on introducing the essential ways of interacting with NoSQL data stores. The types of NoSQL stores vary and so do the ways of accessing and interacting with them. This topic attempts to summarize a few of the most prominent of these disparate ways of accessing and querying data in NoSQL databases.
- **Recap:**
- Revision of Database Management Systems.

Interfacing and Interacting with NoSQL

Objective:

- **In this topic** we focus on MongoDB - Datatypes

1. String – This is the most commonly used datatype to store the data.
2. Integer – This type is used to store a numerical value. ...
3. Boolean – This type is used to store a boolean (true/ false) value.
4. Double – This type is used to store floating point values.

Recap:

- Revision of Nosql Databases.

Data types

- **MongoDB supports many datatypes. Some of them are –**
- **String** – This is the most commonly used datatype to store the data. String in MongoDB must be UTF-8 valid.
- **Integer** – This type is used to store a numerical value. Integer can be 32 bit or 64 bit depending upon your server.
- **Boolean** – This type is used to store a boolean (true/ false) value.
- **Double** – This type is used to store floating point values.
- **Min/ Max keys** – This type is used to compare a value against the lowest and highest BSON elements.
- **Arrays** – This type is used to store arrays or list or multiple values into one key.
- **Timestamp** – ctimestamp. This can be handy for recording when a document has been modified or added.
- **Object** – This datatype is used for embedded documents.
- **Null** – This type is used to store a Null value.
- **Symbol** – This datatype is used identically to a string; however, it's generally reserved for languages that use a specific symbol type.
- **Date** – This datatype is used to store the current date or time in UNIX time format. You can specify your own date time by creating object of Date and passing day, month, year into it.
- **Object ID** – This datatype is used to store the document’s ID.
- **Binary data** – This datatype is used to store binary data.
- **Code** – This datatype is used to store JavaScript code into the document.
- **Regular expression** – This datatype is used to store regular expression.

Data types

Data Model

- BSON format (binary JSON)
- Developers can easily map to modern object-oriented languages without a complicated ORM layer.
- lightweight, traversable, efficient

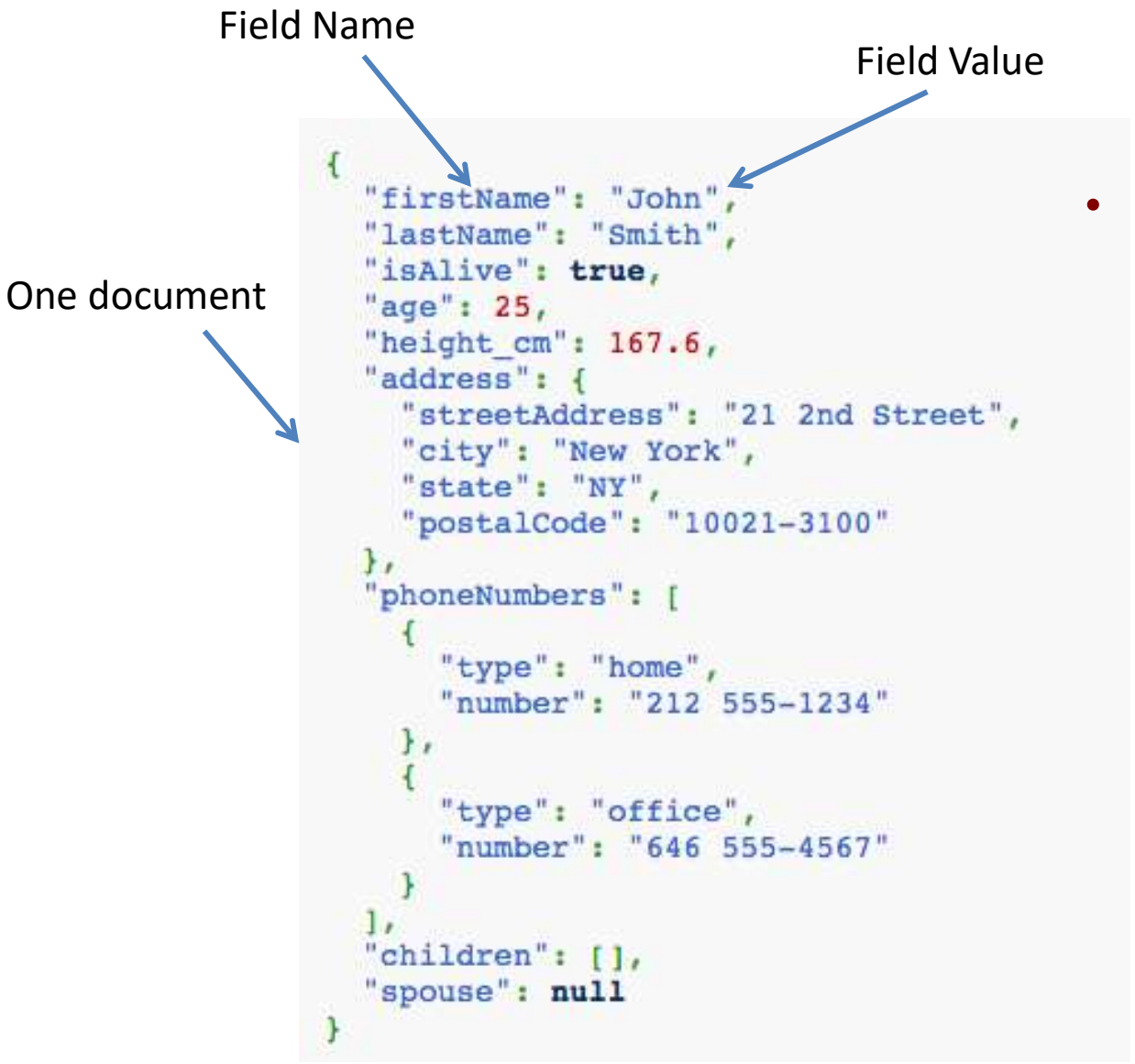
Data types

Terms Mapping (DB vs. MongoDB)

RDBMS	MongoDB
Database	Database
Table	Collection
Tuple/Row	Document
column	Field
Table Join	Embedded Documents
Primary Key	Primary Key (Default key _id provided by mongodb itself)
Database Server and Client	
Mysqld/Oracle	mongod
mysql/sqlplus	mongo

Data types

JSON

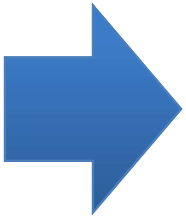


- ***Field Value***
  - Scalar (Int, Boolean, String, Date, …)
  - Document (Embedding or Nesting)
  - Array of JSON objects

Data types

Another Example

```
{ author: 'joe',
  created : new Date('03/28/2009'),
  title : 'Yet another blog post',
  text : 'Here is the text...',
  tags : [ 'example', 'joe' ],
  comments : [
              { author: 'jim',
                comment: 'I disagree'
              },
              { author: 'nancy',
                comment: 'Good post'
              }
  ]
}
```



Remember it is stored in binary formats (BSON)

```
"\x16\x00\x00\x00\x02hello\x00
 \x06\x00\x00\x00world\x00\x00"

"1\x00\x00\x00\x04BSON\x00&\x00
 \x00\x00\x020\x00\x08\x00\x00
 \x00awesome\x00\x011\x00333333
 \x14@\x102\x00\xc2\x07\x00\x00
 \x00\x00"
```

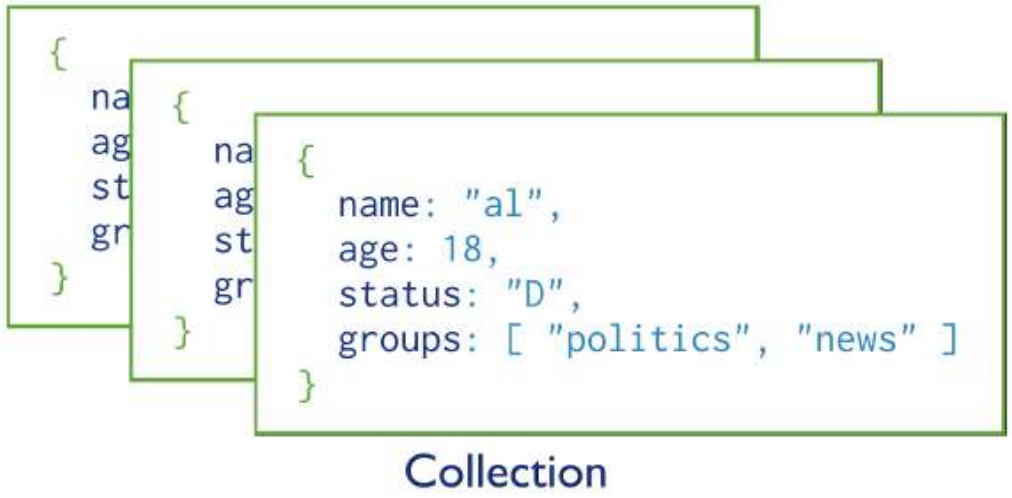
Data types

MongoDB Model

One ***document*** (e.g., one tuple in RDBMS)

```
{
  name: "sue",          <-- field: value
  age: 26,              <-- field: value
  status: "A",          <-- field: value
  groups: [ "news", "sports" ]   <-- field: value
}
```

One ***Collection*** (e.g., one Table in RDBMS)



- **Collection** is a group of similar documents
- Within a collection, each document must have a unique Id

Unlike RDBMS:
No Integrity Constraints in MongoDB

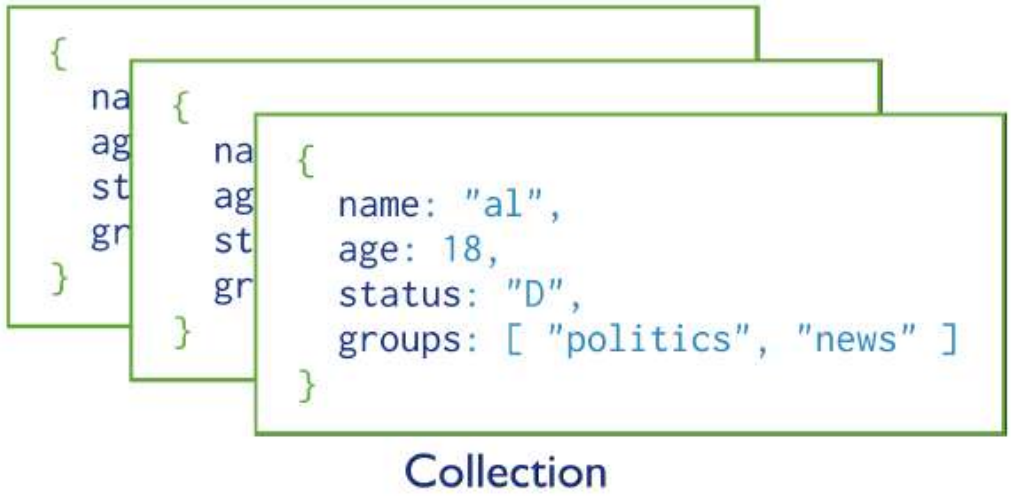
Data types

MongoDB Model

One ***document*** (e.g., one tuple in RDBMS)

```
{
  name: "sue",                    ← field: value
  age: 26,                        ← field: value
  status: "A",                    ← field: value
  groups: [ "news", "sports" ]    ← field: value
}
```

One ***Collection*** (e.g., one Table in RDBMS)



- The field names **cannot** start with the **$** character
- The field names **cannot** contain the **.** character
- Max size of single document 16MB

Data types

```
{
  _id: ObjectId(7df78ad8902c)
  title: 'MongoDB Overview',
  description: 'MongoDB is no sql database',
  by: 'tutorials point',
  url: 'http://www.tutorialspoint.com',
  tags: ['mongodb', 'database', 'NoSQL'],
  likes: 100,
  comments: [
    {
      user:'user1',
      message: 'My first comment',
      dateCreated: new Date(2011,1,20,2,15),
      like: 0
    },
    {
      user:'user2',
      message: 'My second comments',
      dateCreated: new Date(2011,1,25,7,45),
      like: 5
    }
  ]
}
```

Example Document in MongoDB

- _id is a special column in each document
- Unique within each collection
- _id ←→ Primary Key in RDBMS
- _id is 12 Bytes, you can set it yourself
- Or:
  - 1st 4 bytes → timestamp
  - Next 3 bytes → machine id
  - Next 2 bytes → Process id
  - Last 3 bytes → incremental values

NoSQL Storage Architecture

Objective:

- **In this topic** we focus on the NoSQL database approach which is characterized by a move away from the complexity of SQL based servers. The logic of validation, access control, mapping querieable indexed data, correlating related data, conflict resolution, maintaining integrity constraints, and triggered procedures is moved out of the database layer.
- **Recap:**
- Revision of RDBMS architecture.

NoSQL Storage Architecture

- Architecture Pattern is a logical way of categorizing data that will be stored on the Database. NoSQL is a type of database which helps to perform operations on big data and store it in a valid format. It is widely used because of its flexibility and a wide variety of services.

- Architecture Patterns of NoSQL:
- The data is stored in NoSQL in any of the following four data architecture patterns.

- 1. Key-Value Store Database
- 2. Column Store Database
- 3. Document Database
- 4. Graph Database

NoSQL Storage Architecture

- These are explained as following below.

- 1. Key-Value Store Database:
- This model is one of the most basic models of NoSQL databases. As the name suggests, the data is stored in form of Key-Value Pairs. The key is usually a sequence of strings, integers or characters but can also be a more advanced data type. The value is typically linked or co-related to the key. The key-value pair storage databases generally store data as a hash table where each key is unique. The value can be of any type (JSON, BLOB(Binary Large Object), strings, etc). This type of pattern is usually used in shopping websites or e-commerce applications.

- Advantages:
- Can handle large amounts of data and heavy load,
- Easy retrieval of data by keys.
- Limitations:

NoSQL Storage Architecture

- Complex queries may attempt to involve multiple key-value pairs which may delay performance.
- Data can be involving many-to-many relationships which may collide.
- Examples:

- DynamoDB
- Berkeley DB

Key:1	ID:210

Key:2	ID:411	Email: geeksforgeeks@gmail.com

Key:3	UID:219	Name: Geek	Age:20

NoSQL Storage Architecture

- 2. Column Store Database:
- Rather than storing data in relational tuples, the data is stored in individual cells which are further grouped into columns. Column-oriented databases work only on columns. They store large amounts of data into columns together. Format and titles of the columns can diverge from one row to other. Every column is treated separately. But still, each individual column may contain multiple other columns like traditional databases.
- Basically, columns are mode of storage in this type.

- Advantages:

- Data is readily available
- Queries like SUM, AVERAGE, COUNT can be easily performed on columns.
- Examples:

- HBase
- Bigtable by Google
- Cassandra

NoSQL Storage Architecture

Emp ID	Team ID	Emp Name
1	1	Alfie
2	1	Niharika
3	2	Geek

Row-Oriented Database

1	1	Alfie
2	1	Niharika
3	2	Geek

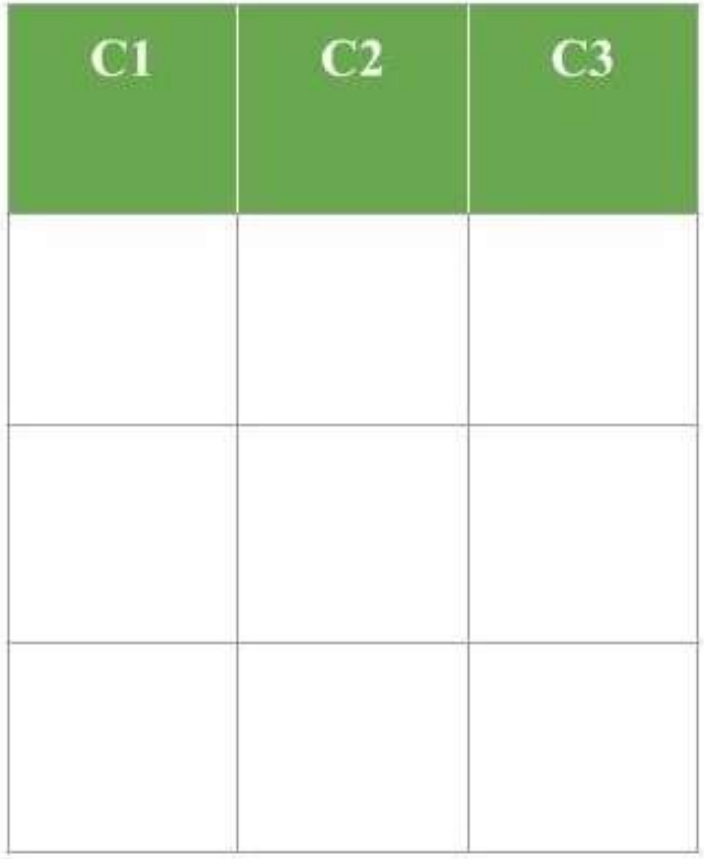
Column Oriented Database

1	2	3
1	1	2
Alfie	Niharika	Geek

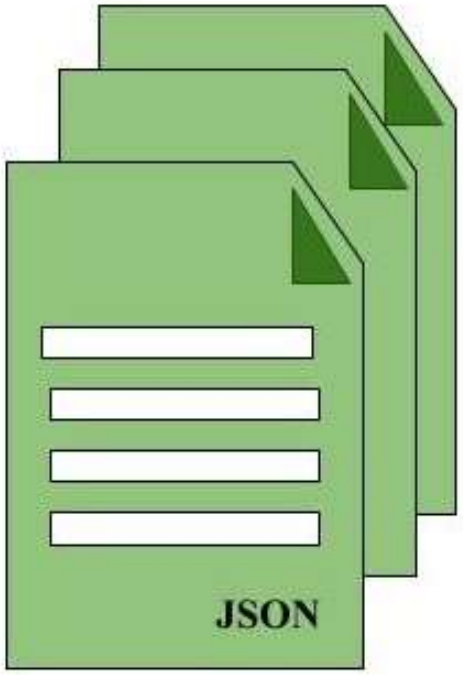
NoSQL Storage Architecture

- 3. Document Database:
- The document database fetches and accumulates data in form of key-value pairs but here, the values are called as Documents. Document can be stated as a complex data structure. Document here can be a form of text, arrays, strings, JSON, XML or any such format. The use of nested documents is also very common. It is very effective as most of the data created is usually in form of JSONs and is unstructured.
- Advantages:
- This type of format is very useful and apt for semi-structured data.
- Storage retrieval and managing of documents is easy.
- Limitations:
- Handling multiple documents is challenging
- Aggregation operations may not work accurately.
- Examples:
- MongoDB
- CouchDB

NoSQL Storage Architecture



Relational Data Model



Document Store Model

Figure – Document Store Model in form of JSON documents

NoSQL Storage Architecture

- 4. Graph Databases:
- Clearly, this architecture pattern deals with the storage and management of data in graphs. Graphs are basically structures that depict connections between two or more objects in some data. The objects or entities are called as nodes and are joined together by relationships called Edges. Each edge has a unique identifier. Each node serves as a point of contact for the graph. This pattern is very commonly used in social networks where there are a large number of entities and each entity has one or many characteristics which are connected by edges. The relational database pattern has tables that are loosely connected, whereas graphs are often very strong and rigid in nature.

- Advantages:

- Fastest traversal because of connections.
- Spatial data can be easily handled.
- Limitations:
- Wrong connections may lead to infinite loops.

- Examples:

- Neo4J
- FlockDB(Used by Twitter)

NoSQL Storage Architecture

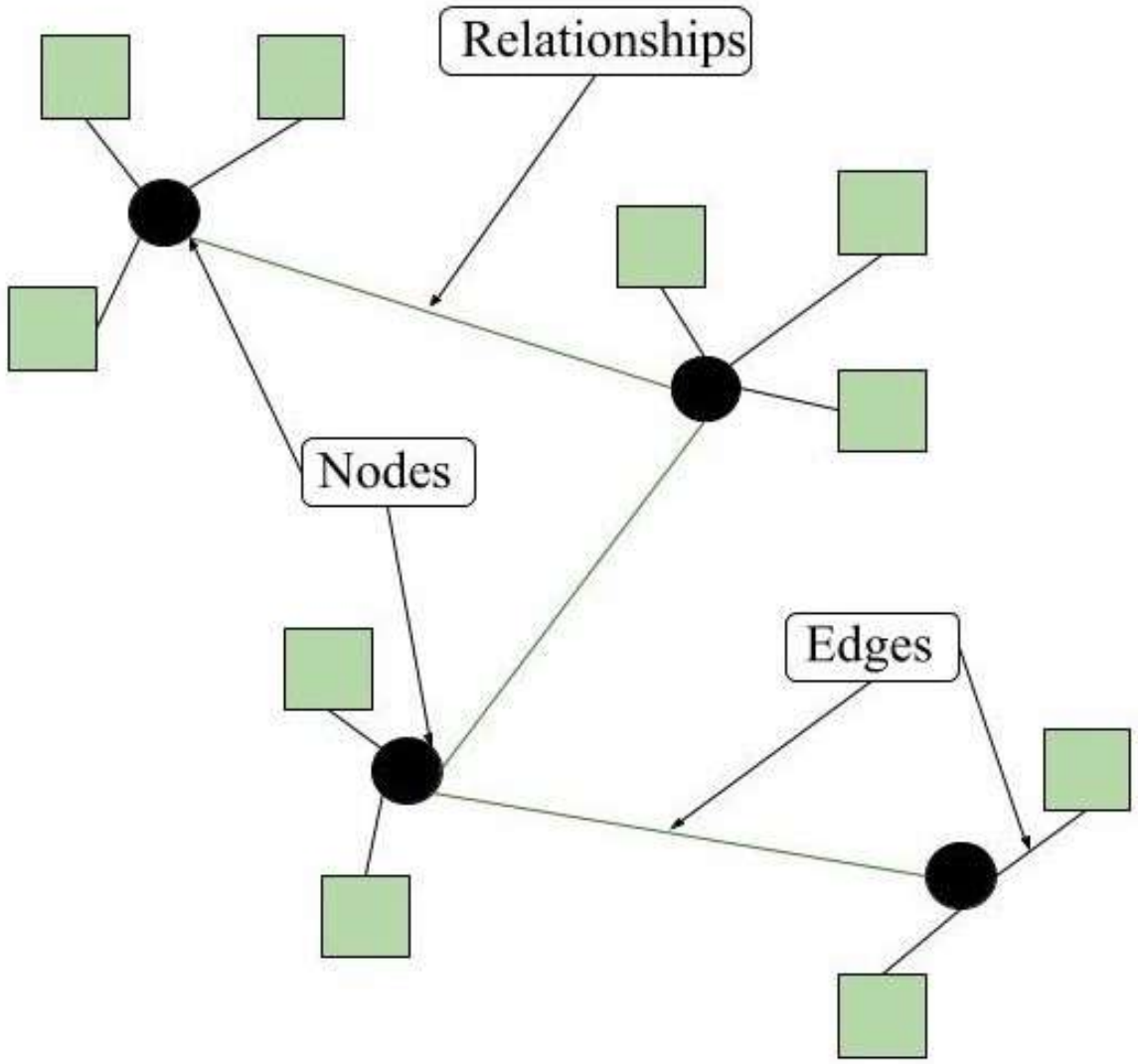


Figure – Graph model format of NoSQL Databases

CRUD operations with MongoDB

Objective:

- **In this topic** we focus on CRUD Meaning: CRUD is an acronym that comes from the world of computer programming and refers to the four functions that are considered necessary to implement a persistent storage application: create, read, update and delete.
- **Recap:**
- Revision of Database Management Systems.

CRUD operations with MongoDB

Must Practice It

Install it

Practice simple stuff

Move to complex stuff

Install it from here: http://www.mongodb.org

Manual: http://docs.mongodb.org/master/MongoDB-manual.pdf
(Focus on Ch. 3, 4 for now)

Dataset: http://docs.mongodb.org/manual/reference/bios-example-collection/

CRUD operations with MongoDB

CRUD

- **C**reate
  - db.collection.insert(<document>)
  - db.collection.save(<document>)
  - db.collection.update(<query>, <update>, { upsert: true })
- **R**ead
  - db.collection.find(<query>, <projection>)
  - db.collection.findOne(<query>, <projection>)
- **U**pdate
  - db.collection.update(<query>, <update>, <options>)
- **D**elete
  - db.collection.remove(<query>, <justOne>)

CRUD operations with MongoDB

CRUD Examples

```
> db.user.insert({
        first: "John",
        last : "Doe",
        age: 39
})
```

```
> db.user.find ()
{
        "_id" : ObjectId("51…"),
        "first" : "John",
        "last" : "Doe",
        "age" : 39
}
```

```
> db.user.update(
        {"_id" : ObjectId("51…")},
        {
            $set: {
                age: 40,
                salary: 7000}
        }
)
```

```
> db.user.remove({
        "first": /^J/
})
```

Querying, Modifying and Managing NoSQL Data stores

Objective:

- **In this topic** we focus on Most NoSQL and NewSQL data stores which implement some sort of horizontal partitioning or sharding, which involves storing sets or rows/records into different segments (or shards) which may be located on different servers.
- **Recap:**
- Revision of Database Management Systems.

Querying, Modifying and Managing NoSQL Data stores

Examples

In RDBMS

```
CREATE TABLE users (
    id MEDIUMINT NOT NULL
        AUTO_INCREMENT,
    user_id Varchar(30),
    age Number,
    status char(1),
    PRIMARY KEY (id)
)
```

```
DROP TABLE users
```

In MongoDB

Either insert the 1st docuement

```
db.users.insert( {
    user_id: "abc123",
    age: 55,
    status: "A"
} )
```

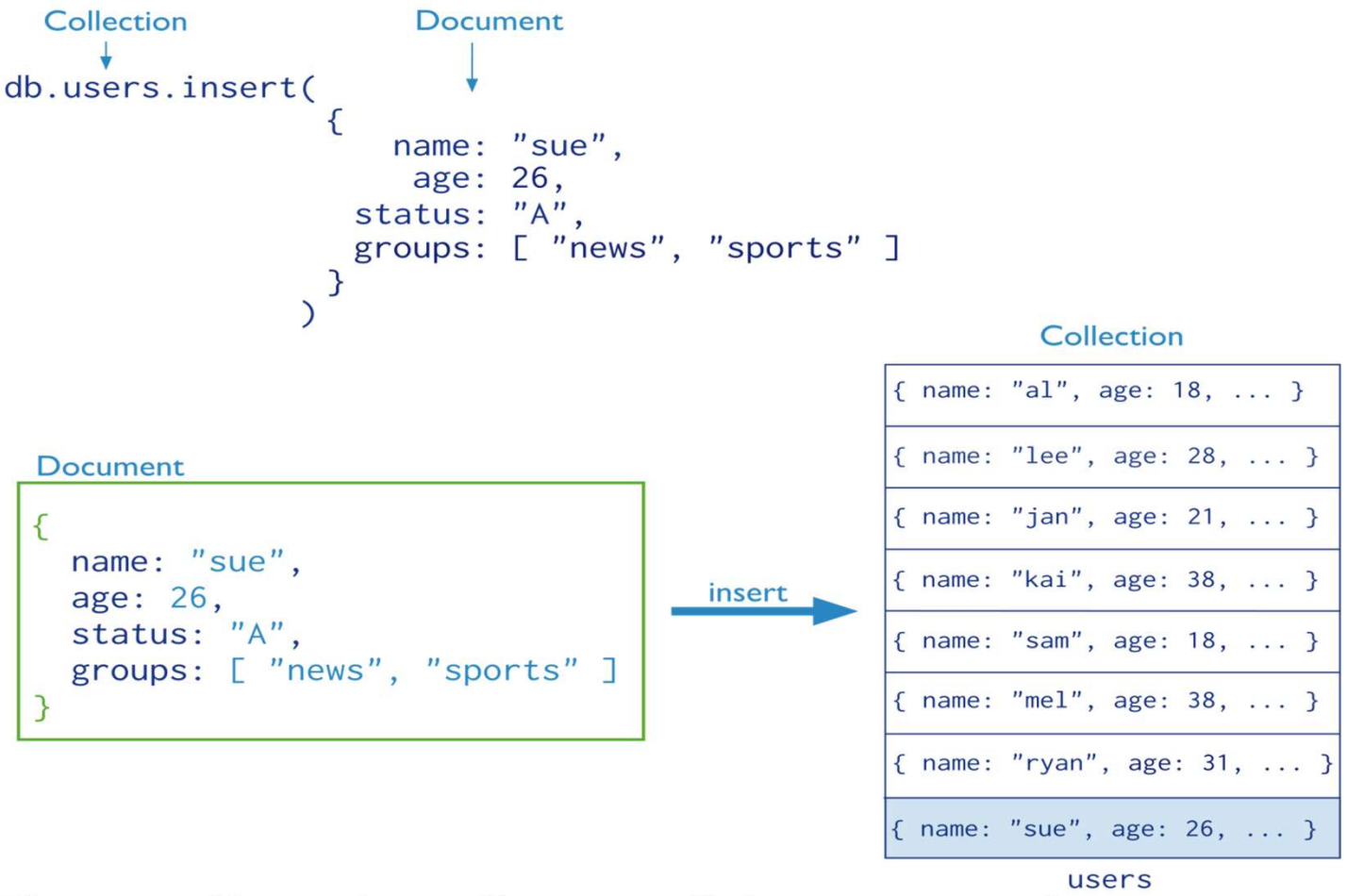
Or create "Users" collection explicitly

```
db.createCollection("users")
```

```
db.users.drop()
```

Creating, Updating and Deleing documents & Querying

Insertion



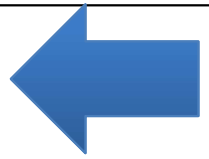
- The collection “users” is created automatically if it does not exist

Creating, Updating and Deleing documents & Querying

Multi-Document Insertion (Use of Arrays)

```
var mydocuments =
    [
      {
        item: "ABC2",
        details: { model: "14Q3", manufacturer: "M1 Corporation" },
        stock: [ { size: "M", qty: 50 } ],
        category: "clothing"
      },
      {
        item: "MNO2",
        details: { model: "14Q3", manufacturer: "ABC Company" },
        stock: [ { size: "S", qty: 5 }, { size: "M", qty: 5 }, { size: "L", qty: 1 } ],
        category: "clothing"
      },
      {
        item: "IJK2",
        details: { model: "14Q2", manufacturer: "M5 Corporation" },
        stock: [ { size: "S", qty: 5 }, { size: "L", qty: 1 } ],
        category: "houseware"
      }
    ];
```

```
db.inventory.insert( mydocuments );
```



All the documents are inserted at once

Creating, Updating and Deleing documents & Querying

Multi-Document Insertion (Bulk Operation)

- A temporary object in memory
- Holds your insertions and uploads them at once

There is also *Bulk Ordered* object

```
var bulk = db.inventory.initializeUnorderedBulkOp();

bulk.insert(
    {
      item: "BE10",
      details: { model: "14Q2", manufacturer: "XYZ Company" },
      stock: [ { size: "L", qty: 5 } ],
      category: "clothing"
    }
);
bulk.insert(
    {
      item: "ZYT1",
      details: { model: "14Q1", manufacturer: "ABC Company"  },
      stock: [ { size: "S", qty: 5 }, { size: "M", qty: 5 } ],
      category: "houseware"
    }
);

bulk.execute();
```

_id column is added automatically

Creating, Updating and Deleing documents & Querying

Deletion
(Remove Operation)

- You can put condition on any field in the document (even ***_id***)

```
db.users.remove(          <-- collection
   { status: "D" }        <-- remove criteria
)
```

The following diagram shows the same query in SQL:

```
DELETE FROM users         <-- table
WHERE   status = 'D'      <-- delete criteria
```

db.users.remove ( ) ➡ Removes all documents from *users* collection

Creating, Updating and Deleing documents & Querying

Update

```
db.users.update(                          <-- collection
   { age: { $gt: 18 } },                  <-- update criteria
   { $set: { status: "A" } },             <-- update action
   { multi: true }                        <-- update option
)
```

Otherwise, it will update only the 1st matching document

Equivalent to in SQL:

```
UPDATE users          <-- table
SET    status = 'A'   <-- update action
WHERE  age > 18       <-- update criteria
```

Creating, Updating and Deleing documents & Querying

Update (Cont'd)

Two operators

```
db.inventory.update(
    { item: "MNO2" },
    {
      $set: {
        category: "apparel",
        details: { model: "14Q3", manufacturer: "XYZ Company" }
      },
      $currentDate: { lastModified: true }
    }
)
```

For the document with `item` equal to `"MNO2"`, use the `$set` operator to update the `category` field and the `details` field to the specified values and the `$currentDate` operator to update the field `lastModified` with the current date.

Creating, Updating and Deleing documents & Querying

Replace a document

```
db.inventory.update(
    { item: "BE10" },
    {
      item: "BE05",
      stock: [ { size: "S", qty: 20 }, { size: "M", qty: 5 } ],
      category: "apparel"
    }
)
```

Query Condition

New doc

For the document having item = “BE10”, replace it with the given document

Creating, Updating and Deleing documents & Querying

Insert or Replace

```
db.inventory.update(
    { item: "TBD1" },
    {
      item: "TBD1",
      details: { "model" : "14Q4", "manufacturer" : "ABC Company" },
      stock: [ { "size" : "S", "qty" : 25 } ],
      category: "houseware"
    },
    { upsert: true }
)
```

The ***upsert*** option

If the document having item = “TBD1” is in the DB, it will be replaced
Otherwise, it will be inserted.

Creating, Updating and Deleing documents & Querying

```
{
   _id: ObjectId(7df78ad8902c)
title: 'MongoDB Overview',
   description: 'MongoDB is no sql database',
by: 'tutorials point',
   url: 'http://www.tutorialspoint.com',
tags: ['mongodb', 'database', 'NoSQL'],
likes: 100,     comments: [
      {
         user:'user1',
         message: 'My first comment',
dateCreated: new Date(2011,1,20,2,15),
like: 0
      },
{
         user:'user2',
         message: 'My second comments',
dateCreated: new Date(2011,1,25,7,45),
like: 5
      }
   ]
}
```

Any relational database has a typical schema design that shows number of tables and the relationship between these tables. While in MongoDB, there is no concept of relationship.

_id is a 12 bytes hexadecimal number which assures the uniqueness of every document. You can provide _id while inserting the document. If you don't provide then MongoDB provides a unique id for every document. These 12 bytes first 4 bytes for the current timestamp, next 3 bytes for machine id, next 2 bytes for process id of MongoDB server and remaining 3 bytes are simple incremental VALUE.

Creating, Updating and Deleing documents & Querying

MongoDB Create Database

The use Command

MongoDB **use DATABASE_NAME** is used to create database. The command will create a new database if it doesn't exist, otherwise it will return the existing database.

Syntax

Basic syntax of **use DATABASE** statement is as follows:

```
use DATABASE_NAME
```

Example

If you want to create a database with name **<mydb>**, then **use DATABASE** statement would be as follows:

```
>use mydb switched
to db mydb
```

To check your currently selected database, use the command **db**

```
>db mydb
```

If you want to check your databases list, use the command **show dbs**.

```
>show dbs local
0.78125GB test
0.23012GB
```

Your created database (mydb) is not present in list. To display database, you need to insert at least one document into it.

```
>db.movie.insert({"name":"tutorials point"})
>show dbs
local       0.78125GB mydb
0.23012GB
```

MongoDB

```
test        0.23012GB
```

In MongoDB default database is test. If you didn't create any database, then collections will be stored in test database.

MongoDB Drop Database

The dropDatabase() Method

MongoDB **db.dropDatabase()** command is used to drop a existing database. **Syntax**

Basic syntax of **dropDatabase()** command is as follows:

```
db.dropDatabase()
```

This will delete the selected database. If you have not selected any database, then it will delete default 'test' database.

Example

First, check the list of available databases by using the command, **show dbs.**

```
>show dbs local      0.78125GB mydb      0.23012GB
test        0.23012GB
>
```

If you want to delete new database **<mydb>**, then **dropDatabase()** command would be as follows:

```
>use mydb switched
to db mydb
>db.dropDatabase()
>{ "dropped" : "mydb", "ok" : 1 }
>
```

Now check list of databases.

```
>show dbs local
0.78125GB test
0.23012GB> In this
chapter, we will see
how to create a
collection using
MongoDB.
```

Creating, Updating and Deleing documents & Querying

MongoDB Create Collection

The createCollection() Method

MongoDB **db.createCollection(name, options)** is used to create collection. **Syntax**

Basic syntax of **createCollection()** command is as follows:

```
db.createCollection(name, options)
```

In the command, **name** is name of collection to be created. **Options** is a document and is used to specify configuration of collection.

Parameter	Type	Description
Name	String	Name of the collection to be created
Options	Document	(Optional) Specify options about memory size and indexing

Options parameter is optional, so you need to specify only the name of the collection. Following is the list of options you can use:

Field	Type	Description
capped	Boolean	(Optional) If true, enables a capped collection. Capped collection is a fixed size collection that automatically overwrites its oldest entries when it reaches its maximum size. **If you specify true, you need to specify size parameter also.**
autoIndexID	Boolean	(Optional) If true, automatically create index on _id field. Default value is false.
size	number	(Optional) Specifies a maximum size in bytes for a capped collection. **If capped is true, then you need to specify this field also.**

Creating, Updating and Deleing documents & Querying

max	number	(Optional) Specifies the maximum number of documents allowed in the capped collection.

While inserting the document, MongoDB first checks size field of capped collection, then it checks max field.

Examples

Basic syntax of **createCollection()** method without options is as follows:

```
>use test switched
to db test
>db.createCollection("mycollection")
{ "ok" : 1 }
>
```

You can check the created collection by using the command **show collections**.

```
>show collections
mycollection system.indexes
```

The following example shows the syntax of **createCollection()** method with few important options:

```
>db.createCollection("mycol", { capped : true, autoIndexID : true, size : 6142800,
max : 10000 } )
{ "ok" : 1 }
>
```

In MongoDB, you don't need to create collection. MongoDB creates collection automatically, when you insert some document.

```
>db.tutorialspoint.insert({"name" : "tutorialspoint"})
>show collections
mycol mycollection
system.indexes
tutorialspoint
>
```

Indexing and ordering datasets (MongoDB)

Objective:

- **In this topic** we focus on MongoDB uses multikey indexes to index the content stored in arrays. When you index on a column that holds an array value, MongoDB creates separate index entries for every element of the array. These multikey indexes allow queries to select documents that contain arrays by matching on element or elements of the arrays.

Recap:

- Revision of DBMS architecture.

Indexing and ordering datasets (MongoDB)

- Indexes support the efficient resolution of queries. Without indexes, MongoDB must scan every document of a collection to select those documents that match the query statement. This scan is highly inefficient and require MongoDB to process a large volume of data.

- Indexes are special data structures, that store a small portion of the data set in an easy-to-traverse form. The index stores the value of a specific field or set of fields, ordered by the value of the field as specified in the index.

- The createIndex() Method
- To create an index, you need to use createIndex() method of MongoDB.

- Syntax
- The basic syntax of createIndex() method is as follows().

- >db.COLLECTION_NAME.createIndex({KEY:1})
- Here key is the name of the field on which you want to create index and 1 is for ascending order. To create index in descending order you need to use -1.

Indexing and ordering datasets (MongoDB)

- Example
- >db.mycol.createIndex({"title":1})
- {
- "createdCollectionAutomatically" : false,
- "numIndexesBefore" : 1,
- "numIndexesAfter" : 2,
- "ok" : 1
- }
- >
- In createIndex() method you can pass multiple fields, to create index on multiple fields.

- >db.mycol.createIndex({"title":1,"description":-1})
- >

Indexing and ordering datasets (MongoDB)

Parameter	Type	Description
background	Boolean	Builds the index in the background so that building an index does not block other database activities. Specify true to build in the background. The default value is **false**.
unique	Boolean	Creates a unique index so that the collection will not accept insertion of documents where the index key or keys match an existing value in the index. Specify true to create a unique index. The default value is **false**.
name	string	The name of the index. If unspecified, MongoDB generates an index name by concatenating the names of the indexed fields and the sort order.
sparse	Boolean	If true, the index only references documents with the specified field. These indexes use less space but behave differently in some situations (particularly sorts). The default value is **false**.
expireAfterSeconds	integer	Specifies a value, in seconds, as a TTL to control how long MongoDB retains documents in this collection.
weights	document	The weight is a number ranging from 1 to 99,999 and denotes the significance of the field relative to the other indexed fields in terms of the score.
default_language	string	For a text index, the language that determines the list of stop words and the rules for the stemmer and tokenizer. The default value is **English**.
language_override	string	For a text index, specify the name of the field in the document that contains, the language to override the default language. The default value is language.

Indexing and ordering datasets (MongoDB)

- The dropIndex() method
- You can drop a particular index using the dropIndex() method of MongoDB.
- Syntax
- The basic syntax of DropIndex() method is as follows().
- >db.COLLECTION_NAME.dropIndex({KEY:1})
- Here key is the name of the file on which you want to create index and 1 is for ascending order. To create index in descending order you need to use -1.
- Example
- > db.mycol.dropIndex({"title":1})
- {
- "ok" : 0,
- "errmsg" : "can't find index with key: { title: 1.0 }",
- "code" : 27,
- "codeName" : "IndexNotFound"
- }
- The dropIndexes() method
- This method deletes multiple (specified) indexes on a collection.
- Syntax
- The basic syntax of DropIndexes() method is as follows() −

Indexing and ordering datasets (MongoDB)

- >db.COLLECTION_NAME.dropIndexes()
- Example
- Assume we have created 2 indexes in the named mycol collection as shown below −
- > db.mycol.createIndex({"title":1,"description":-1})
- Following example removes the above created indexes of mycol −
- >db.mycol.dropIndexes({"title":1,"description":-1})
- { "nIndexesWas" : 2, "ok" : 1 }
- >The getIndexes() method
- This method returns the description of all the indexes int the collection.
- Syntax
- Following is the basic syntax od the getIndexes() method −
- db.COLLECTION_NAME.getIndexes()
- Example
- Assume we have created 2 indexes in the named mycol collection as shown below −

- > db.mycol.createIndex({"title":1,"description":-1})

Cloud database: - Introduction of Cloud database

Objective:

- **In this topic** we focus on cloud database which is a database service built and accessed through a cloud platform. It serves many of the same functions as a traditional database with the added flexibility of cloud computing. Users install software on a cloud infrastructure to implement the database.
- **Recap:**
- Revision of Cloud Architecture.

Cloud database: - Introduction of Cloud database

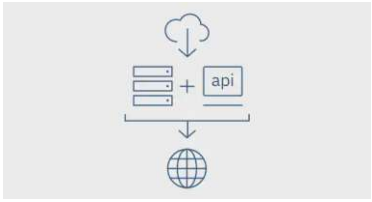
- A cloud database is a database service built and accessed through a cloud platform. It serves many of the same functions as a traditional database with the added flexibility of cloud computing. Users install software on a cloud infrastructure to implement the database.

- Key features:

- A database service built and accessed through a cloud platform
- Enables enterprise users to host databases without buying dedicated hardware
- Can be managed by the user or offered as a service and managed by a provider
- Can support relational databases (including MySQL and PostgreSQL) and NoSQL databases (including MongoDB and Apache CouchDB)
- Accessed through a web interface or vendor-provided API

Cloud database: - Introduction of Cloud database

- **Why cloud databases**



- **Ease of access**
- Users can access cloud databases from virtually anywhere, using a vendor's API or web interface.

- **Scalability**
- Cloud databases can expand their storage capacities on run-time to accommodate changing needs. Organizations only pay for what they use.

- **Disaster recovery**
- In the event of a natural disaster, equipment failure or power outage, data is kept secure through backups on remote servers.

Cloud database: - Introduction of Cloud database

- **Considerations for cloud databases**
- **Control options**

- Users can opt for a virtual machine image managed like a traditional database or a provider's database as a service (DBaaS).

- **Database technology**

- SQL databases are difficult to scale but very common. NoSQL databases scale more easily but do not work with some applications.

- **Security**

- Most cloud database providers encrypt data and provide other security measures; organizations should research their options.

- **Maintenance**

- When using a virtual machine image, one should ensure that IT staffers can maintain the underlying infrastructure.

Cloud database: - Introduction of Cloud database

- **WHAT IS A CLOUD DATABASE?**
- let's dig deeper into the cloud-based world that we are living in. So, cloud database services include everything from storing all kinds of data required to providing access and delivering the data to the required parties involved. Therefore, as mentioned above, it is storing the data on the internet and is normally of three kinds.

- Platform as a service(PaaS)

- Software as a service(SaaS)

- Infrastructure as a service(IaaS)

- Platform as a service or PaaS is the most common type here, providing the provision of servers, data storage, and operating systems. It helps in the storage and acts as a platform for the virtual database, saving the hardware cost and helping to access the data from all around the world.

- SaaS, on the other hand, provides the entire software as a service to the organization in exchange for an amount and is an excellent business option for all those organizations involving a lot of web users.

- IaaS helps to provide a complete infrastructure where the business can run their applications.

Cloud database: - Introduction of Cloud database

CLOUD DATABASE TECHNOLOGIES LIST

- CLOUCloud computing is on a rise because of the flexibility and the ease of services that it provides. Several well-known IT giants are planning to capture the market. Most of the cloud databases run on the well-known cloud computing platforms like Rackspace, salesforce, GoGrid, and Amazon EC2.

- **Here are the top five most beneficial cloud services for data storage.**

- Amazon Web Services or AWS- AWS needs no introduction as it is already counted as one of the top cloud database technologies.
- Azure by Microsoft- This is Microsoft's entry into the cloud space which has already gained a lot of momentum.
- Oracle Database cloud- Everyone has heard about Oracle because of its traditional database system, and now it is capturing the cloud storage space.
- SAP- SAP is the giant when it comes to offering software for enterprises and now is ready for cloud storage with its platform called HANA.D DATABASE TECHNOLOGIES LIST

NoSQL with Cloud Database

Objective:

- **In this topic** we focus on NoSQL databases are specifically designed for low cost commodity hardware. These databases are mostly used for storage and access of data across multiple storage cluster. For example Google, Facebook, Google+, Google big table, Amazon Dynamo, Twitter etc. collects and stores Terabytes of data for their user every day.
- **Recap:**
- Revision of Cloud Architecture.

NoSQL with Cloud Database

- **What is a cloud database?**
- A “cloud database” can be one of two distinct things: a traditional or NoSQL database installed and running on a cloud virtual machine (be it public cloud, private cloud, or hybrid cloud platforms), or a cloud provider’s fully managed database-as-a-service (DBaaS) offering. The former, running your own self-managed database in a cloud environment, is really no different from operating a traditional database. Cloud DBaaS, on the other hand, is the natural database equivalent of software-as-a-service (SaaS): pay as you go, and only for what you use, and let the system handle all the details of provisioning and scaling to meet demand, while maintaining consistently high performance.
- **Cloud database options:**
- Traditional database running on cloud virtual machine (VM)
- Fully managed database-as-a-service

Most of the time (and for most of the remainder of this page), the term “cloud database” refers to a cloud-based database-as-a-service.

NoSQL with Cloud Database

- **Why use a cloud database/DBaaS?**
- The key benefits of cloud databases are that they are accessible from anywhere, scalable from day one, and designed for reliability and performance.
- **Common cloud database use cases**
- Cloud databases work in most cases that traditional databases do. They are particularly valuable when building software products that:
- ***cloud-native [***Cloud-native technologies empower organizations to build and run scalable applications in modern, dynamic environments such as public, private, and hybrid clouds.]
- ***Require large volume of data***
- ***Need to handle high scale traffic***
- ***Are distributed geographically***
- Data applications that take advantage of centralization, like analytics, are also fantastic candidates for cloud database usage.
- While certain use cases are more obvious candidates for cloud database usage, more traditional use cases, like real-time online transaction processing, caching, or data warehousing work just as well in the fully managed paradigm.

NoSQL with Cloud Database

- Cloud database considerations
- Whether you're still thinking about whether a cloud database is right for you, or in the process of selecting the ideal database-as-a-service for your needs, there are a few key factors to take into consideration:

- Cloud Database Providers
- Database Technology
- Management System
- Cost Model
- Security

NoSQL with Cloud Database

- **MongoDB Atlas cloud database**

MongoDB can be installed and run on any cloud provider or on-premise network as a self-managed database cluster or virtual machine, or on AWS, or Azure using MongoDB Atlas, our cloud database-as-a-service (DBaaS) offering. There are major benefits to adopting the DBaaS option, including:

- **Simplified management**
- **Elastic autoscaling-Atlas** uses to **automatically scale** your cluster tier, storage capacity, or both in response to cluster usage.
- **Redundancy, backup, and restore**
- **Charts** -Atlas Charts offers a quick, simple, and powerful way to visualize your data from Atlas and Atlas Data Lake. Atlas Data Lake allows you to natively query and combine data across MongoDB Atlas and AWS S3 without complex integrations.
- **Connectors**-The MongoDB Connector for Business Intelligence for Atlas (**BI Connector**) is only available for M10 and larger clusters. The BI Connector is a powerful tool which provides users SQL-based access to their MongoDB databases. As a result, the BI Connector performs operations which may be CPU and memory intensive.
- **Schema navigator-**The **Schema** tab provides an overview of the data type and shape of the fields in a particular collection.

NoSQL with Cloud Database

- MongoDB Atlas, part of MongoDB's broader data-as-a-service (DaaS) development platform, is a powerful and compelling alternative to managing your own NoSQL, or traditional, database, or using a cloud provider-specific managed offering.

- The way a cloud database works is that rather than installing, configuring, and maintaining a database instance or instances, an automated system is able to provision, manage, and scale the underlying database cluster for you.

- Fully managed database services handle the complexities of maintaining a consistently available, high performance cluster in a way that allows you, the developer, to access it as a simple, globally available resource.

- You can treat the cluster as a single database instance, covered by a transparent usage-based pricing model, so you're never worrying about over- or under-provisioning.

NoSQL with Cloud Database

- MongoDB Stitch was designed to:
  ✅ **Reduce backend complexity** by handling database interactions and authentication.
  ✅ **Enable serverless computing** with built-in cloud functions.
  ✅ **Integrate with MongoDB Atlas** for seamless database connectivity.
- It allowed developers to build **scalable applications without managing backend infrastructure** by providing:
- **Easy access to MongoDB Atlas** without writing backend code.
- **Flexible authentication** (Google, Facebook, Apple, JWT, API Keys).
- **Cloud Functions** to run custom logic in response to events.
- **GraphQL & REST APIs** for fetching and modifying data.
- **Triggers** to automate workflows based on database changes.

NoSQL with Cloud Database

- **Key Features of MongoDB Stitch**
- **Authentication & Access Control**
  - Supports **OAuth providers** (Google, Facebook, Apple).
  - Custom authentication with JWT and API keys.
  - **Role-based access control (RBAC)** to protect sensitive data.
- **Stitch Functions (Serverless Computing)**
  - Run JavaScript functions in the cloud **without managing servers**.
  - Triggered by database changes, HTTP requests, or scheduled events.
- **Stitch Triggers**
  - React to **database events in real-time**.
  - Example: Notify users when a document is inserted or updated.
- **REST & GraphQL APIs**
  - Expose MongoDB data as an **API without additional setup**.
  - GraphQL support allows fetching **only the necessary data**, improving efficiency.
- **Integration with Third-Party Services**
  - Connect with AWS, Twilio, Stripe, Slack, and other APIs.

NoSQL with Cloud Database

- **What is MongoDB Cloud Manager?**
- MongoDB **Cloud Manager** is a **cloud-based management platform** for **monitoring, automating, and backing up MongoDB clusters**. It helps teams **deploy, manage, and scale MongoDB databases** while ensuring high availability and security.
- **Think of it as a MongoDB-specific DevOps tool** that provides **automation, monitoring, and backup** for on-premise or self-managed cloud deployments.

NoSQL with Cloud Database

- **Uses of MongoDB Cloud Manager**

A. Monitoring MongoDB Clusters

- Provides **real-time performance metrics** (CPU, memory, queries per second).
- Customizable **alerts** for performance issues and failures.
- Tracks **replication lag**, **index usage**, and **query execution times**.

B. Automated Deployment & Scaling

- **One-click cluster provisioning** for MongoDB deployments.
- Automates **sharding, replica sets, and configuration updates**.
- **Auto-healing**: Replaces failed nodes automatically.

C. Continuous Backups & Point-in-Time Recovery

- **Automated snapshots** of MongoDB data.
- **Point-in-time restores** to recover from accidental data loss.
- Supports **multiple backup strategies** (incremental and full backups).

D. Security & Access Management

- **Role-Based Access Control (RBAC)** for granular permissions.
- **End-to-End Encryption** to protect sensitive data.
- **Audit logging** to track database changes.

E. Integration with Third-Party Tools

- Connects with **Slack, PagerDuty, and Datadog** for alerting.
- Exports logs to **Splunk, Prometheus, and Grafana** for analysis.

NoSQL with Cloud Database

Difference: MongoDB Cloud Manager vs. Atlas

Feature	MongoDB Cloud Manager	MongoDB Atlas
Database Hosting	Self-managed (on-prem or cloud)	Fully managed (MongoDB-as-a-Service)
Automation	Manual setup required	Automatic
Backup	Continuous backup available	Built-in backups
Scaling	Manual or scripted scaling	Auto-scaling
Best for	Teams managing self-hosted MongoDB	Companies needing a fully managed database

NoSQL with Cloud Database

- **Key Benefits of MongoDB Cloud Manager**

Simplifies MongoDB Operations – Automates cluster management.
Enhances Security – Built-in authentication, encryption, and access control.
Improves Performance – Real-time monitoring and query optimization.
Ensures Data Safety – Continuous backups with fast recovery.

Daily Quiz

Q1. Compare NoSQL & RDBMS

Q2. What is NoSQL?

Q3. What are the features of NoSQL?

Q4. Explain the difference between NoSQL v/s Relational database?

Q5. Explain “Polyglot Persistence” in NoSQL?

Q6. How does NoSQL DB budget memory?

Q7. How to script NoSQL DB configuration?

Q8. Does NoSQL Database Interact With Oracle Database?

Q9. What is the difference between NoSQL & Mysql DBs’?

Q10. Explain Oracle NoSQL database?

NIET Greater Noida
Autonomous Institute
Get Future Ready

Weekly Assignemnt

1. _____ is a online NoSQL developed by Cloudera.
   (a) HCatalog
   (b) Hbase
   (c) Imphala
   (d) Oozie
2. Which of the following is not a NoSQL database?
   (a) SQL Server
   (b) MongoDB
   (c) Cassandra
   (d) None of the mentioned
3. Which of the following is a NoSQL Database Type?
   (a) SQL
   (b) Document databases
   (c) JSON
   (d) All of the mentioned
4. Which of the following is a wide-column store?
   (a) Cassandra
   (b) Riak
   (c) MongoDB
   (d) Redis
5. "Sharding" a database across many server instances can be achieved with _____
   (a) LAN
   (b) SAN
   (c) MAN
   (d) All of the mentioned

Assignment

Q1: What are NoSQL databases? What are the different types of NoSQL databases?

Q2: What do you understand by NoSQL databases? Explain.

Q3: Explain difference between scaling horizontally and vertically for databases

Q4: What are the advantages of NoSQL over traditional RDBMS?

Q5: When should we embed one document within another in MongoDB?

Q6: Define ACID Properties?

Q7: Does MongoDB support ACID transaction management and locking functionalities?

Q8: Explain advantages of BSON over JSON in MongoDB?

Q9: How can you achieve primary key - foreign key relationships in MongoDB?

Q10: How do I perform the SQL JOIN equivalent in MongoDB?

MCQ

- 1. Most NoSQL databases support automatic _____ meaning that you get high availability and disaster recovery.
- (a) processing
- (b) scalability
- (c) replication
- (d) all of the mentioned
-
- 2. Which of the following are the simplest NoSQL databases?
- (a) Key-value
- (b) Wide-column
- (c) Document
- (d) All of the mentioned

- 3._____ stores are used to store information about networks, such as social connections.
- (a) Key-value
- (b) Wide-column
- (c) Document
- (d) Graph

MCQ

- 4. NoSQL databases is used mainly for handling large volumes of _____ data.
- (a) unstructured
- (b) structured
- (c) semi-structured
- (d) all of the mentioned
-
- 5. Which of the following language is MongoDB written in?
- (a) Javascript
- (b) C
- (c) C++
- (d) All of the mentioned

- 6. Point out the correct statement.
- (a) MongoDB is classified as a NoSQL database
- (b) MongoDB favors XML format more than JSON
- (c) MongoDB is column-oriented database store
- (d) All of the mentioned

MCQ

- 7. Which of the following format is supported by MongoDB?
- (a) SQL
- (b) XML
- (c) BSON
- (d) All of the mentioned

- 8. NoSQL was designed with security in mind, so developers or security teams don't need to worry about implementing a security layer. Is it true or false?
- (a) True
- (b) False

- 9. Which of the following is not a reason NoSQL has become a popular solution for some organizations?
- (a) Better scalability
- (b) Improved ability to keep data consistent
- (c) Faster access to data than relational database management systems (RDBMS)
- (d) More easily allows for data to be held across multiple servers

- 10. NoSQL prohibits structured query language (SQL). Is it True or False?
- (a) True
- (b) False

Glossary Questions

Fill the following blanks with one of the given options-

Glossary Based Questions:

1. (a) Horizontally scalable (b) Vertically scalable (c) Sharding (d) They don't scale
   i. SQL databases are such that_____.
   ii. _____ means scaling by adding more machines to your pool of resources (also described as "scaling out").
   iii. _____ refers to scaling by adding more power (e.g., CPU, RAM) to an existing machine (also described as "scaling up").
   iv. _____is a method of splitting and storing a single logical dataset in multiple databases.
2. (a) Database (b) Field (c) Document (d) Collection
   i. _____is a type of nonrelational database that is designed to store and query data as JSON-like documents.
   ii. _____database (MongoDB) collects and processes analytics data from all your websites.
   iii. A database _____ is a set of data values, of the same data type, in a table.
   iv. _____an organized collection of structured information, or data, typically stored electronically in a computer system.
3. (a) CouchDB (b) MongoDB (c) HBase (d) PostgreSQL
   i. _____is an open-source non-relational distributed database modelled after Google's Bigtable and written in Java.
   ii. _____is also a clustered database that allows you to run a single logical database server on any number of servers or VMs.
   iii. _____is used as the primary data store or data warehouse for many web, mobile, geospatial, and analytics applications.
   iv. _____is a source-available cross-platform document-oriented database program. Classified as a NoSQL database program.

Expected Questions for University Exam

1. What do you mean by NoSQL?
2. What are the features of NoSQL?
3. What is the CAP theorem? How is it applicable to NoSQL systems?
4. Explain the difference: RDBMS vs NoSQL?
5. What are the major challenges with traditional RDBMS?
6. What are the different types of NoSQL databases?
7. How Does NoSQL relate to big data?
8. Can you explain the transaction support by using a BASE in NoSQL?
9. What is a Key-Value store or Key-Value database?
10. What is the Column store database?

References

Text books:
1) Korth, Silbertz, Sudarshan," Database System Concepts", Seventh Edition, McGraw - Hill.
2) Elmasri, Navathe, " Fundamentals of Database Systems", Seventh Edition, Addision Wesley.
3) Ivan Bayross "SQL,PL/SQL The programming language Oracle, Forth Edition, BPB Publication.
Reference Books:
1) Thomas Cannolly and Carolyn Begg, "Database Systems: A Practical Approach to Design, Implementation and Management", Third Edition, Pearson Education, 2007.
2) Raghu Ramakrishan and Johannes Gehrke "Database Management Systems" Third Edition, McGraw-Hill.
3) NoSQL and SQL Data Modeling: Bringing Together Data, Semantics, and Software First Edition by Ted Hills.
4) Brad Dayley "NoSQL with MongoDB in 24 Hours" First Edition, Sams Publisher.

Recap

- This unit provide us fundamentals domain of NOSQL and its latest trends in industry.
- In this unit we are also benefitted with the knowledge of different types of databases in NOSQL.
- Whether you experience a natural disaster, power failure or other crisis, having your data stored in the cloud ensures it is backed up and protected in a secure and safe location. Being able to access your data again quickly allows you to conduct business as usual, minimizing any downtime and loss of productivity
- This unit will impart us with knowledge of Cloud Databases and querying on cloud databases.

Faculty Video Links, You tube & NPTEL Video Links and Online Courses Details

You Tube video

http://www.nptelvideos.com/lecture.php?id=6516

http://www.nptelvideos.com/lecture.php?id=6517

http://www.nptelvideos.com/lecture.php?id=6518

http://www.nptelvideos.com/lecture.php?id=6519

https://www.youtube.com/watch?v=2yQ9TGFpDuM

Thank You